

NEAR BY
PROJECT REPORT

Submitted by

AHMED ZIYAD K(CSAXBCA022)
ANANDU M C(CSAXBCA024)
MOHAMMED BILAL T P(CSAXBCA032)
MUHAMMED SINAN K P(CSAXBCA040)

In Partial fulfilment of the requirements for the
award of the degree of
BACHELOR OF COMPUTER APPLICATION
AT



MES COLLEGE OF ARTS AND SCIENCES,
CHATHAMANGALAM,
CALICUT 673601
(Affiliated to university of Calicut)
2022-2025

MES COLLEGE OF ARTS AND SCIENCES
CHATHAMANGALAM
(Affiliated to university of Calicut)



CERTIFICATE

This is to certify that the project entitled "NEAR BY" is bona fide record of the work carried out by AHMED ZIYAD K(CSAXBCA022), ANANDU M C (CSAXBCA024), MUHAMMED MOHAMMED BILAL T P

(CSAXBCA032), MUHAMMED SINAN K P (CSAXBCA040) student of Bachelor of computer application, MES COLLEGE OF ARTS AND SCIENCES, CHATHAMANGALAM, KERALA during the year 2025 partial fulfillment for the award of the degree of BACHELOR OF COMPUTER APPLICATION of Calicut University and that project has not formed the basis for the award previously of any degree diploma, associateship, fellowship, or any other similar title

PROJECT COORDINATOR
Mrs.MOHSINA

HEAD OF DEPARTMENT
Mr. THAHSEN M

(Dept Office Seal)

Internal Examiner

External Examiner

MES COLLEGE OF ARTS AND SCIENCES
CHATHAMANGALAM
(Affiliated to university of Calicut)



DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that the project entitled “**NEAR BY**” is bona fide work of **AHMED ZIYAD K (CSAXBCA022), ANANDU M C (CSAXBCA024), MOHAMMED BILAL T P (CSAXBCA032), and MUHAMMED SINAN K P (CSAXBCA040)**, carried out as part of the project work under my supervision.

PROJECT GUIDE
Mrs. DEEPTHI P

DECLARATION

I hereby declare that this project report entitled “NEAR BY” submitted for the Bachelor of computer application degree is our original and project has not formed the basis for the award of any degree, associateship, fellowship or any other similar titles

AHMED ZIYAD K(CSAXBCA022)
ANANDU M C(CSAXBCA024)
MOHAMMED BILAL T P(CSAXBCA032)
MUHAMMED SINAN K P(CSAXBCA040)

Place: KALANTHODE

Date:

ACKNOWLEDGEMENT

The satisfaction and euphoria of the successful completion of any task would be incomplete without the mention of the people whom made possible, whose content guidance and encouragement crowded our effort with success.

I have been fortunate enough to be able to secure co-operation, guidance and assistance from a number of people. We are at loss of how to express the deep sense of gratitude that we have towards all of them. We are greatly indebted to our Principal Prof. SHAFEEKALATHUR who has given the permission for the fulfilment of this venture.

I would like to express my gratitude to Mr. THAH SIN M head of the Department of Computer Science, for his valuable support throughout this project and also to be a contolead us in the right path throughout our careers in the college.

I would like to express my gratitude to Mrs. MOHSINAMISS lecture, Department of Computer Science, for his valuable support throughout this project and also to be a contolead us in the right path throughout our careers in the college.

I would like to express our sincere thanks to the god almighty for this constant love and grace that he has bestowed upon me finally, we thank my parent, family members, and our beloved friends for their moral support and encouragement without which we would not have been able to follow our dreams.

NEAR BY

ABSTRACT

NEARBY

This project presents the design and implementation of a mobile-based local shopping and delivery application named NearBy, developed to provide fast and convenient access to products available in nearby local shops. The system reduces the need for customers to physically visit multiple stores or wait for long delivery times offered by traditional e-commerce platforms. By enabling same-day delivery and direct shop pickup, the application ensures time-efficient and user-friendly shopping.

The NearBy application connects customers, local shop owners, and delivery agents through a single digital platform. Customers can search for products, view shop locations, place orders, make secure payments, and track deliveries in real time. Shop owners can register, manage their products and stock, process orders, view payments, and interact with customers through ratings, reviews, and complaints. Delivery agents are responsible for collecting and delivering orders within nearby locations, ensuring fast and reliable service.

The system is developed using Flutter for cross-platform mobile application development and Django (Python) for backend processing. It includes GPS-based location tracking, notification management, secure authentication, and role-based access control. The application supports local businesses by improving visibility and increasing sales while enhancing customer convenience and promoting efficient local commerce..

Modules:

1. ADMIN:

- Login
- Change password
- Verify Shop (Approve/Reject)
- View Accepted Shops
- View Rejected Shops
- Verify Delivery Boy (Approve/Reject)
- View Accepted Delivery Boys
- View Rejected Delivery Boys
- View Customer
- View Application feedback

2. SHOP:

- Sign up
- Login
- View / Edit Profile
- Manage Categories
- Manage Product
- Stock Management
- View Damaged Product
- View Order & Assign Delivery
- View Delivery Status
- View Payment
- View Rating & Review
- Manage Offer

- View Complaint & Send Replies

- Manage Notification
- Change Password

3.USER:

- Sign up
- Login
- Change Password
- View & Edit Product
- View Product
- Add to Cart
- View Cart and Make Payment
- View Previous Order
- Return Damaged Product
- Send Rating & Review
- Send Complaint & View Replies
- View Notification
- View Offer
- Send Feedback

4. DELIVER BOY

- Registration
- Login
- Change Password
- View and Edit Profile
- View Allocated Orders
- Update Delivery Status

CONTENT

<u>CHAPTERS</u>	<u>PAGE</u>
1.INTRODUCTION	1
1.1 about thesystem	2
2. SYSTEM ANALYSIS	4
2.1 Existing System	4
2.2 Proposed System	4
3.FEASIBILITY STUDY	7
3.1 Technical Feasibility	8
3.2 Economical Feasibility	8
3.3 Operational Feasibility	8
3.4 Behavioral Feasibility	9
4. SOFTWARE ENGINEERINGPARADIGRAM	11
4.1 Agile Model	11
4.2 Scrum	14
5. SOFTWARE REQUIREMENT SPECIFICATION	16
5.1 System Requirements	17
5.2 Frontend	18
5.3 Backend	21
5.4 Database	22
5.5 Software's	23
6.SYSTEM DESIGN	27
6.1 Input Design	28
6.2 Output Design	29
Normalization	30
6.3 Modules and Description	32
6.4 Data Flow Diagram	33

6.6 Entity Relationship Diagram	39
6.7 Class Diagram	41
6.8 Activity Diagram	44
6.9 Data Design	49
7. SYSTEM TESTING AND IMPLEMENTATION	54
7.1 System Testing	55
7.2 System Implementation	58
8. SYSTEM MAINTENANCE	62
8.1 Corrective Maintenance	63
8.2 Adaptive Maintenance	63
8.3 Perfective Maintenance	63
8.4 Preventive Maintenance	63
9.FUTURE ENHANCEMENT	66
10.CONCLUSION	68
11.BIBLOGRAPHY	70
12. APPENDIX	71
12.1 Sample Screenshot	72
12.2 Sample Code	75

INTRODUCTION

1.1.About the system

Near By

NearBy is a mobile-based local shopping and delivery application designed to provide quick and convenient access to products available in nearby local shops. The system aims to overcome the limitations of traditional shopping methods, where customers must physically visit multiple stores to check product availability, which is time-consuming and inefficient. By digitizing local commerce, NearBy enables users to find required products easily and place orders from their smartphones.

The application allows customers to browse products from registered nearby shops, view shop locations, compare prices, and choose between home delivery or direct shop pickup. This flexibility helps users save time while meeting urgent product needs such as groceries, medicines, and household items. Real-time order tracking and secure payment options further enhance the overall user experience.

NearBy also plays a vital role in empowering local shop owners by providing them with a digital platform to showcase their products, manage stock, process orders, and receive customer feedback. Without investing in complex infrastructure, small businesses can expand their reach and increase sales within their locality.

In addition, the system integrates delivery agents who are responsible for collecting and delivering orders efficiently. This creates flexible employment opportunities while ensuring fast and reliable deliveries. By combining customers, shops, and delivery agents into a single platform, NearBy promotes smarter local shopping, supports local businesses, and improves service efficiency through modern mobile and web technologies.

SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

System analysis is a general term that refers to an orderly, structured process for identifying and solving a problem. The system analysis process is called the life cycle methodology, since it relates to four significant phases in the life cycle of all business information systems: study, design, development and operation. The definition of system analysis includes not only the process but also the process of putting together to form a new system. A system analyst is an individual who performs system analysis during any, or all, of the life cycle phases of a business information system. The system analyst not only analyses business information system problems, but also synthesizes new solutions to solve those problems to meet other information needs.

The various techniques used in the study of the present system are:

- Observation
- Interviews
- Site visits
- Discussion

2.1. Existing system

In the existing system, customers rely on traditional shopping methods or large online marketplaces to purchase products. In physical shopping, customers must visit multiple nearby shops to check product availability, which is time-consuming and inconvenient. There is no prior information about stock availability, leading to unnecessary travel and effort. This method is inefficient for customers who require products urgently.

Although online shopping platforms such as large e-commerce websites provide a wide variety of products, they usually require two to three days for delivery. These platforms are designed for large-scale distribution and do not focus on local shops or immediate delivery needs. As a result, customers cannot access nearby products quickly, and local shop owners often remain excluded from digital marketplaces..

2.2. Proposed system

The proposed system introduces a mobile-based local shopping and delivery application named NearBuy, designed to provide a faster and more efficient alternative to traditional shopping methods and large online marketplaces. The system connects customers with nearby local shops, allowing users to search for products, check availability, and place orders through a single mobile application. This approach reduces the need to physically visit multiple shops and helps customers save time and effort.

NearBuy allows users to choose between home delivery and direct shop pickup, making it suitable for urgent product requirements such as groceries, medicines, and household items. The application provides secure digital payment options and notification features to ensure a smooth and reliable shopping experience. These features improve convenience and make the overall purchasing process simpler and more accessible. The system also supports local shop owners by offering a digital platform to manage product listings, stock levels, and customer orders efficiently. In addition, delivery agents are integrated into the system to ensure timely delivery of orders within nearby locations, while creating flexible employment opportunities. Overall, the proposed system enhances customer satisfaction, supports local businesses, and promotes efficient local commerce through modern mobile technology.

FEASIBILITY STUDY

3. FEASIBILITY STUDY

A feasibility study is a preliminary study undertaken to determine and document a project's viability. The results of this study are used to decide whether to proceed with the project. It is an analysis of possible alternative solutions to a problem and recommendation on the best alternative. The feasibility study proposes one or more conceptual solutions to the problem set for the project. The conceptual solution gives an idea of what the new system will look like. The feasibility study environment enables all alternatives to be discussed and evaluated. This phase starts with an identification of the main characteristics of the required system. During this stage it is important to collect as much information as possible about the software package that might meet the specification from as many sources as possible. Normally, feasibility study is a cost benefit analysis of various alternatives. It can be defined as a systematic comparison between the cost of carrying out a service or activity and the value of that service or activity. The main benefits are qualitative rather than quantitative. A feasibility study could be used to test a new working system, which could be used because:

- The current system may no longer suit its purpose
- Technological advancement may have rendered the current system obsolete
- The business is expanding allowing it to cope with extra workload
- Customer are complaining about the speed and quality of work the business provides.

Feasibility study is carried out to determine whether the proposed system is possible to develop with available resources and what should be the cost consideration. Factors considered in the feasibility analysis are:

- Technical Feasibility
- Economic Feasibility
- Operational Feasibility
- Behavioral Feasibility

3.1. Technical Feasibility

Technical Feasibility study is performed to check whether the proposed system is technically Feasible or not. Technical feasibility center son the existing computer system (hardware, software.) and to what extent it can support the proposed addition. This involves financial consideration to accommodate technical enhancement. This system is technically feasible. All the data is stored in files. The input can be done through dialog boxes which are both interactive and user friendly. Hard copies can be obtained for future use, by diverting the documents to a printer. Windows serves as the platform for the new system. This project was developed using the Flutter framework which is technically feasible as it requires only the existing technology.

3.2. Economic Feasibility

Economic Feasibility Study is the most frequently used method for evaluating the effectiveness of a candidate system. More commonly known as cost/benefit analysis, the procedure is to determine the benefits and savings that are expected from a candidate system and compare them with cost. This analysis phase determines how much cost is needed to produce the proposed system. This system is economically feasible since it does not require any initial setup. It requires only network connection and the cost needed to host the site.

3.3. Operational Feasibility

Operational Feasibility study is performed to check whether the system is operationally Feasible or not. Using command buttons throughout the application programs enhances operational feasibility. So, maintenance and modification are found to be easier. The purpose of this feasibility is to determine whether the new system will be used if it is developed and implemented. Usually, people are reluctant to change their progression. Hence

an additional Effort is to be made to train and educate the user so then way of the system. In this project the system is well coded and designed such that when it is compiled it is well developed and installed. We can do maintenance and modifications easily. There is no difficulty in implementing the system and the proposed system is user friendly and functionally reliable and thus it is operational feasible.

3.4. Behavioral Feasibility

This analysis involves how it will work when it is installed and the assessment of the political and managerial environment in which it is implemented. People are inherently resistant to change and computers have been known to facilitate change. An estimate should badious the reaction of the user staff towards the development of the computerized system. The introduction of a candidate system requires special effort to educate, sell and train the staff for conducting thebusiness. This includes the following questions:

- Is there sufficient support for the users?
- Will the proposed system cause harm?
- This system is behavior feasible in the system sufficiently supports the user sand the system does not cause any harm to the users.

The projectwould be beneficial because it satisfies the objectives when developedandinstalled. All the behavioral aspects are considered carefully and concludethatthe project is behaviorally feasible.

SOFTWARE ENGINEERING PARADIGM

4.SOFTWARE ENGINEERING PARADIGM

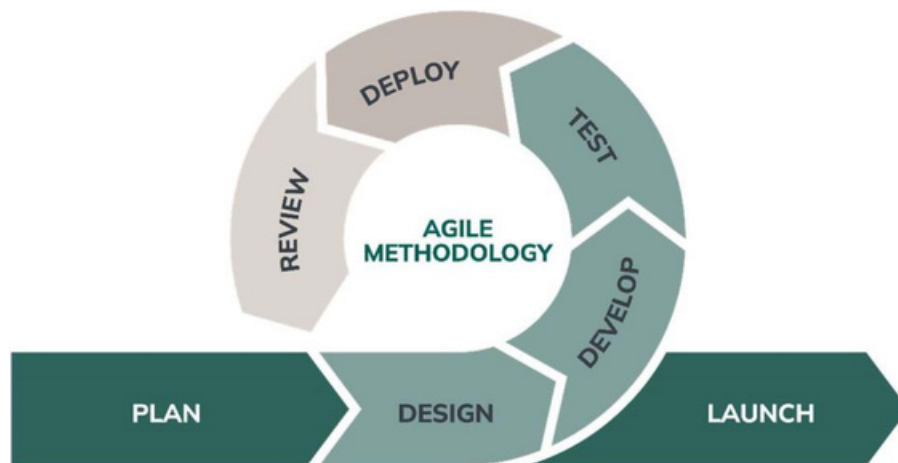
The software engineering paradigm which is also referred to as a software process model or software development life cycle (SDLC model is the development strategy that encompasses the process, method and tools. SDLC describes the period of that time starts with the software system being conceptualized.

4.1. AGILE MODEL

Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapidly very of working software products. Agile Methods break the product into small incremental builds. These builds are provided in iterations. Each iteration typically lasts from about one to three weeks. Every iteration involves cross functional teams working simultaneously on various areas. The Agile model believes that every project needs to be handled differently and the existing methods need to be tailored to best suit the project requirements. In Agile, the tasks are divided into time boxes (small time frames) to deliver specific features for a release. Iterative approach is taken and working software build is delivered after each iteration. Each build is incremental in terms of features; the final build holds all the features required by the customer.

Graphical illustration of the Agile Model:

Here are the important stages involved in the Agile Model process in the SDLC lifecycle:



Requirements gathering and analysis

In this phase, you must define the requirements. You should explain business opportunities and plan the time and effort needed to build the project. Based on this information, you can evaluate technical and economic feasibility.

Design the requirements

When you have identified the project, work with stakeholders to define requirements. You can use the user flow diagram or the high-level UML diagram to show the work of new features and show how it will apply to your existing system.

Construction/iteration

When the team defines the requirements, the work begins. Designers and developers start working on their project, which aims to deploy a working product. The product will undergo various stages of improvement, so it includes simple, minimal functionality.

Testing

In this phase, the Quality Assurance team examines the product's performance

and looks for the bug. **Deployment** In this phase, the team issues a product for the user's work environment. **Feedback** After releasing the product, the last step is feedback. In this, the team receives feedback about the product and works through the feedback.

Deployment

In this phase, the team issues a product for the user's work environment. **Feedback** After releasing the product, the last step is feedback. In this, the team receives feedback about the product and works through the feedback.

Feedback

After releasing the product, the last step is feedback. In this, the team receives feedback about the product and works through the feedback.

Advantages of Agile Model

- ☐ Is A very realistic approach to software development.
- ☐ Promotes team work and cross training.
- ☐ Functionality can be developed rapidly and demonstrated.
- ☐ Resource requirements are minimum.
- ☐ Suitable for fixed or changing requirements.
- ☐ Delivers early partial working solutions.
- ☐ Minimal rules, documented.
- ☐ Enables concurrent development and delivery within an overall planned context
- ☐ Little or no planning required.
- ☐ Easy to manage.

Disadvantages of Agile Model

- Not suitable for handling complex dependencies.
- More risk of sustainability, maintainability and extensibility.
- An overall plan, an agile leader and agile practice is a must without which it will not work.
- Strict delivery management dictates the scope, functionality to be delivered, and adjustments to meet the deadlines.
- Depends heavily on customer interaction, so if customer is not clear, team can be driven in the wrong direction.

4.2. Scrum

Scrum is an agile process most commonly used for product development. Especially software development. Scrum is a project management framework that is applicable to any project with aggressive deadlines, complex requirements and a degree of uniqueness. In Scrum, projects move forward via series of iterations called sprints. Each sprint is typically two to four weeks long. An introduction to Scrum would not be complete without knowing the Scrum terms you'll be using. This section in the Scrum overview will discuss common concepts in Scrum. Scrum team: A typical scrum team has between five and nine people, but Scrum projects can easily scale into the hundreds. However, Scrum can easily be used by one person teams and often is. This team does not include any of the traditional software engineering roles such as programmer, designer, tester or architect. Everyone on the project works to gather to complete the set of work they have collectively committed to complete within a sprint. Scrum teams develop a deep form of camaraderie and a feeling that "we're all in this together."

SYSTEM REQUIREMENTS SPECIFICATION

5.1. SYSTEM REQUIREMENTS

Hardware Requirements

The selection of hardware is very important in the existence and proper working of any software. Then selection hardware, the size and capacity requirements are also important.

Processor	: i3 or above
System bus	: 32Bit or 64Bit
Hard disk	: 320 GB and above
RAM	: 4 GB and above
Monitor	: SVGA colour
Keyboard	: 104 keys
Mouse	: Optical/scroll mouse

Software Requirements

One of the most difficult tasks is selecting software for the system, once the system requirements are found out then we have to determine whether a particular software package fits for those system requirements. The application requirement

Operating System	: Windows 8 or above
Front End	: HTML, CSS, Javascript, bootstrap, dart
Back End	: Python (Django)

Data Base	: MySQL
Software Used	: PyCharm, Android studio
Mobile Application Framework	: Flutter (for Android app development)

5.2. Frontend

Flutter

Flutter is an open-source mobile application development framework created by Google. It was first introduced in 2017 and has gained popularity among mobile developers due to its ease of use and efficiency in building cross platform applications for both Android and iOS devices. Flutter uses a programming language called Dart, which is also developed by Google, and has its own set of widgets and libraries that enable developers to build beautiful and responsive user interfaces. Flutter's hot reload feature allows developers to see the changes they make to their code in real-time, making the development process faster and more efficient. One of the key advantages of Flutter is that it provides a single codebase for both Android and iOS platforms, reducing the time and effort needed to develop and maintain separate applications for each platform. Additionally, Flutter apps can offer high performance and smooth animations, which can enhance the user experience. Flutter also provides a range of features and tools for developers, such as debugging, testing, and integration with other development tools.

Dart

Dart is a client-optimized programming language for apps on multiple platforms. It is developed by Google and is used to build mobile, desktop, server, and web applications. Dart is an object-oriented, class-based, garbage

collected language with C-style syntax. Dart can compile to either native code or JavaScript. It supports interfaces, mixings, abstract classes, reified generics, and type inference.

CSS

CSS stands for Cascading Style Sheets. It is a language that describes how HTML elements are displayed on a web page. CSS can control the layout, colors, fonts, animations, transitions, and other aspects of the presentation layer. CSS can be written in external files, embedded in HTML head tags, or applied inline to specific elements. CSS follows a set of rules and selectors to determine which styles are applied to which elements. CSS is one of the core technologies of web development, along with HTML and JavaScript.

HTML

Hypertext Markup Language (HTML) is the standard markup language for creating web pages and web applications. With Cascading Style Sheets (CSS) and JavaScript it forms a triad of cornerstone technologies for the World Wide Web. Web browsers receive HTML documents from a web server or from local storage and render them into multimedia web pages. HTML describes the structure of a web page semantically and originally included cues for the appearance of the document.

JAVASCRIPT

JavaScript is a versatile, high-level programming language primarily used for web development. It enables dynamic content, allowing developers to create interactive websites and applications. JavaScript runs on the client side, meaning it executes directly in the user's web browser, which helps reduce server load and enhance user experience. Its capabilities extend beyond the browser, as it can be used on servers through environments like Node.js.

5.3. Back End

PYTHON

Python is a general-purpose language. It has a wide range of applications from Web development (like: Django and Bottle), scientific and mathematical computing (Orange, SymPy, NumPy) to desktop graphical user Interfaces (Pygame, Panda3D). The syntax of the language is clean and the length of the code is relatively short. It's fun to work in Python because it allows you to think about the problem rather than focusing on the syntax.

Python 3.5 is the last updated version and was on September 13, 2015.

Features Of Python Programming

- A simple language which is easier to learn
- Free and open-source
- Portability
- Extensible and Embedded
- A high-level, interpreted language
- Object-oriented

DJANGO (FRAMEWORK)

Django is a high-level, open-source web framework written in Python that promotes rapid development and clean, pragmatic design. It follows the model-template-views (MTV) architectural pattern and includes numerous built-in features such as an ORM (Object Relational Mapping), authentication, and an admin interface, which streamline the development process. Django emphasizes reusability, modularity, and the principle of "don't repeat yourself" (DRY), allowing developers to create robust and scalable web applications.

quickly and efficiently. With its comprehensive documentation and vibrant community, Django is a popular choice for both beginners and experienced developers.

5.4. DATABASE

MySQL

MySQL, the most popular Open-Source SQL database management system, is developed, distributed, and supported by Oracle Corporation. The SQL part of “MySQL” stands for “Structured Query Language”. SQL is the most common standardized language used to access databases. Depending on your programming environment, you might enter SQL directly, embed SQL statements into code written in another language, or use a language-specific API that hides the SQL syntax. SQL is defined by the ANSI/ISO SQL Standard. The SQL standard has been evolving since 1986 and several versions exist.

Features of MySQL:

- MySQL is a database management system.
- MySQL databases are relational.
- MySQL software is Open Source. The MySQL Database Server is very fast, reliable, scalable, and easy to use

SQL Commands

SQL commands are instructions, coded into SQL statements, which are used to communicate with the database to perform specific tasks, work, functions and queries with data. SQL commands can be used not only for searching the database but also to perform various other functions like, creating tables, adding values to tables, or modify data, drop the table, set permissions for users. SQL commands are grouped into four major categories depending on their

functionality. These SQL commands are used for managing changes affecting the data. These commands are COMMIT, ROLLBACK, and SAVEPOINT.

Data Definition Language (DDL)

These SQL commands are used for creating, modifying, and dropping the structure of database objects. The commands are CREATE, ALTER, DROP, RENAME, and TRUNCATE. Data Manipulation Language (DML) These SQL commands are used for storing, retrieving, modifying, and deleting data. These Data Manipulation Language commands are: SELECT, INSERT, UPDATE, and DELETE.

Transaction Control Language (TCL)

These SQL commands are used for managing changes affecting the data. These commands are COMMIT, ROLLBACK, and SAVEPOINT.

Data Control Language (DCL)

These SQL commands are used for providing security to database objects. These commands are GRANT and REVOKE.

5.5. SOFTWARE'S

PyCharm

PyCharm is an advanced Integrated Development Environment (IDE) developed by JetBrains, specifically designed for Python development. It offers intelligent code assistance, automated refactoring, and a powerful debugger, making it ideal for both beginners and experienced developers. PyCharm supports web development with frameworks like Django and Flask, and it integrates seamlessly with front-end technologies such as HTML, CSS, and JavaScript. The IDE includes built-in database tools for MySQL, PostgreSQL,

and SQLite, allowing developers to manage and execute SQL queries directly within the environment. It also features robust version control integration with Git, making collaboration and code management easier. PyCharm simplifies virtual environment and package management, ensuring smooth dependency handling. Available on Windows, macOS, and Linux, PyCharm comes in two editions: a free Community Edition with essential features and a Professional Edition with advanced capabilities such as database support and scientific tools. Its extensive functionality makes it a preferred choice for Python developers working on web applications, machine learning, and data science projects.

Android Studio

Android Studio is the official Integrated Development Environment (IDE) for Android app development, developed by Google. It is based on JetBrains' IntelliJ IDEA and provides a powerful environment for building, testing, and debugging Android applications efficiently. Android Studio offers a wide range of features that streamline the development process. It includes a flexible Gradle-based build system, which allows developers to manage dependencies, compile code, and create multiple build variants with ease. The IDE also provides an advanced code editor with intelligent code completion, real-time error checking, and refactoring tools to enhance productivity. A key feature of Android Studio is its Android Emulator, which enables developers to test applications on different virtual devices with varying screen sizes, resolutions, and Android versions without requiring physical hardware. This helps in faster debugging and optimization of the app.

Android Studio also integrates Layout Editor, a drag-and-drop interface for designing UI elements visually. The built-in APK Analyzer helps optimize the app size and performance, while the Profiler tools allow monitoring CPU, memory, and network usage to enhance app efficiency. With built-in support for Kotlin, Java, and C++, Android Studio provides flexibility in choosing a

preferred programming language. Additionally, it supports Firebase integration, making it easier to incorporate cloud-based features like real-time databases, authentication, and push notifications. Android Studio remains the primary tool for Android developers due to its extensive features, continuous updates, and strong community support, ensuring smooth and efficient Android app development.

SYSTEM DESIGN

6. SYSTEM DESIGN

Daigneault-step process that focuses on data structure, software architecture, procedural details and interface between the modules. The design process also translates the requirements into their presentation n of the software that can access for quality before coding begins. Design is the only way that can accurately translate a customer's finished software product. The most creative and challenging phase of the system development is the system design. It provides the understanding and procedural details necessary for implementing the system recommended in the feasibility study. Design goes through the logical and physical stages of development. System design goes through two phases of development:

- Logical design
- Physical design

Logical Design

The part of the design process that is in dependent of any specific hardware or software platform is referred to as logical design. During logical design, all functional features of the system chosen for development in the analysis phase are described independently of any computer platform. Logical design concentrates on the business aspects of the system and tends to be oriented to a high level of specificity. During the logical design of the proposed system, all the functional features of the system which are identified during the analysis phase are collected and properly specified independent of any hardware and software specifications.

Physical Design

During physical design the logical specifications of the system from logical design are transferred into technology specific details from which all programming and system construction can be accomplished.

6.1. Input Design

In input design, user originated inputs are converted into computer-based format. In output design, the emphasis is on producing the hardcopy of the information requested or displaying the output on the screen in a predefined format. The following features have been incorporated into the input design of the proposed system.

Easy Data Input

Appropriate messages are provided in the message area, which prompts the user to enter the right data. Erroneous data inputs are checked at the end of each screen entry.

Data Validation

The input data is validated to minimize errors in data entry. For certain, data specific codes have been given and validation is done which enables the user to enter the required data or correct them if they entered wrong inputs

User Friendliness

User is never left in a state of confusion as to what is happening, instead appropriate error and acknowledge messages are sent. Bootstrap's alert messages are being used to indicate the error, warning messages.

Consistent Format

A fixed format is adopted for displaying the alert messages. Every screen has a line, which displays the operation that can be performed after the data entry. They are normally done at the touch of a key.

Interactive Dialogue

The system engages the user in an interactive dialogue. The system is able to extract missing or omitted information from the user by directing the user through appropriate messages, which are displayed.

6.2. Output Design

Output design generally refers to their Sult sand information data generated by the system. For many end users, output is the main reason for developing the system and the basis on which they evaluate the usefulness of application. The objective of a system finds its shape in terms of the output. The analysis of the objective of a system leads to determination of outputs. Outputs of a system can take various forms. The most common are reports, screen displays, printed form, graphical drawings etc. The output also varies in terms of their contents, frequency, timing and format. The output from a system is the justification for its existence. If the outputs are inadequate in any way, the system itself is inadequate. The basic requirement of output is that it should be accurate, timely and appropriate in terms of content, medium and layout for its intended purpose. Hence it is necessary to design output, so that the objective of the system is met in the best possible manner. When designing output, the system analyst must accomplish things like, to determine what information to be present, to decide whether to display or print the information and select the output medium to distribute the output to intended recipients. External outputs

are those, whose destination will be outside the organization and which requires special attention as the project image of the organization. Internal output is those whose destination within the organization. It is to be carefully designed, as they are the user's main interface with the system. Interactive outputs are those which the user uses to communicate directly with the computer.

6.3. Normalization

Normalization is the process of decomposing a set of relations with anomalies to produce smaller and well-structured relations that contain minimum redundancy. It is a formal process of deciding which attributes should be grouped together in a relation. The purpose of normalization is to make table as simple as possible. Normalization carried out in the system for following reasons.

- To structure the data. So that there is no repetition of data, this helps in saving space.
- To permit simple retrieval of data in respond set query and request.
- To simplify maintenance of data through updates, insert on and deletion.
- To reduce the need to restructures or recognize data which new application requirements arise.

Primary key is assigned for this purpose. The primary key fields in almost all tables help to ease the search and improve efficiency.

The different normal forms applied during the database design are given below.

- >First Normal Form (1NF)
- >Second Normal Form (2NF)
- >Third Normal Form(3NF)

FIRST NORMAL FORM

First normal form decomposes all data into two dimensional records. First normal form is achieved when all repeating groups are removed so that a record is of fixed length.

SECONDNORMALFORM

Second normal form eliminates any relationships in which data elements do not fully dependent on the primary key of the record. Second normal form is achieved when are cord is of INF and each item in the record is fully functionally depending on the primary key for identification.

THIRDNORMALFORM

Third normal form eliminates any relationships that contain transitive X ahold in Radiotin X, the neither X is super key for R or A is prime (1.e.A is dependencies' relational R is said to be in 3NF, if whenever a functional dependency, subset of the candidate key). In our project the coding is done using python. This language is more powerful.

6.4. Modules & Description

1. ADMIN:

- Login
- Change password
- Verify Shop (Approve/Reject)
- View Accepted Shops
- View Rejected Shops
- Verify Delivery Boy (Approve/Reject)
- View Accepted Delivery Boys

- View Rejected Delivery Boys

View Customer View

Application feedback

2. SHOP:

- Sign up
- Login
- View / Edit Profile
- Manage Categories
- Manage Product
- Stock Management
- View Damaged Product
- View Order & Assign Delivery
- View Delivery Status
- View Payment
- View Rating & Review
- Manage Offer
- View Complaint & Send Replies
- Manage Notification
- Change Password

3.USER:

- Sign up
- Login
- Change Password
- View & Edit Product
- View Product
- Add to Cart

- View Cart and Make Payment
- View Previous Order
- Return Damaged Product
- Send Rating & Review
- Send Complaint & View Replies
- View Notification
- View Offer
- Send Feedback

4.Delivery Boy

- Registration
- Login
- Change Password
- View and Edit Profile
- View Allocated Orders
- Update Delivery Statuts

6.5. Data flow diagram

The data base may be defined as an organized collection of related information. The organized information serves as a base from which further recognition can be retrieved desired information or processing the data. The most important aspect of building an application system is the design of tables. The data flow diagram is used for classifying system requirements for major transformation that will become programs in system design. This is the starting point of the design phase that functionally decomposes the required specifications down to the lower level of details. It consists of a series of bubbles joined together by lines.

- Bubbles: Represent the data transformations.
- Lines: Represents the logic flow of data.

Data can trigger event and can be processed to use full information. System analysis recognizes the central goal of data in organizations. This dataflow analysis tells a great deal about how organization objectives are accomplished. Dataflow analysis studies the use of data in each activity. It documents this finding in DFD's. Dataflow analysis gives the activities so system from the view point of data where it originates, how they are use do range do where they go, including the stops along the way from their destination. The components of dataflow strategy span both requirements determination and system's design. The first part is called dataflow analysis. As the name suggests, we didn't use the data flow analysis tools exclusively for the analysis stage but also in the designing phase with documentation.

Constructing a DFD

Several rules of thumb are used in drawing DFDs: -

- Processes should be named and numbered for easy reference.
- The direction of flow is from source to destination, although they may flow back to a source. One way to indicate this is to draw a long flow line back to the source. An alternative way is to repeat the source symbol as a destination.
- When a process is exploded into lower-level details, they are numbered.
- A level 0 DFD, so-called a context level, represents the entire software elements as a single bubble with input and output indicated by incoming and outgoing arrows respectively. Additional process and information flow parts are represented in the next level i.e. Level 1 DFD. Any process, which is complex in Level 1, will be further represented in to sub functions in the next level, i.e. Level 2 DFD is a means of

Representing a system at any level of detail with a graphic network of symbols showing data flows, data stores, data process, sources or destination. The DFD is designed to aid communication. DFD shows the minimum contents of data stores. In order to show what happens with in a given process, then the detailed explosion of that process is shown. The DFD methodology is quite effective, especially when the required design is unclear and the user and the analyst need a notational language for communication.

Context Diagram

The top-level diagram is often called a "context diagram". It contains a single process, but it plays a very important role in studying the current system. The context diagram defines the system that will be studied in the sense that it determines the boundaries. Dataflow Diagrams are made up of a number of symbols which represent system components. Dataflow modelling uses four kinds of symbols.

Process

Process shows work of the system. Each process has one or more data input and produces one or more data outputs. Processes are represented by circles in DFDs.

Data Stores

Data stores are the repositories of data. Processes can enter data into the store or retrieve the data from the data store. Data stores are represented by parallel lines which may be depicted horizontally or vertically.

Data Flows

The arrows represent dataflow. A dataflow is data in motion. Dataflow

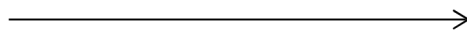
represents an input of data to a process or the output of the data from a process.

External Entities

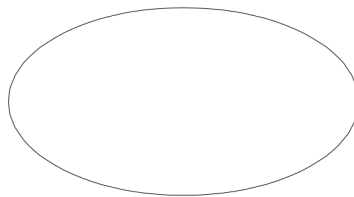
External entities are outside the system but they either supply input to the system or use other system outputs. They are entities on which the designer has control. External entities that supply data into the system are sometimes called source. External entities that use the system are called sinks, represented by rectangles.



: External Entity



: Data Flow

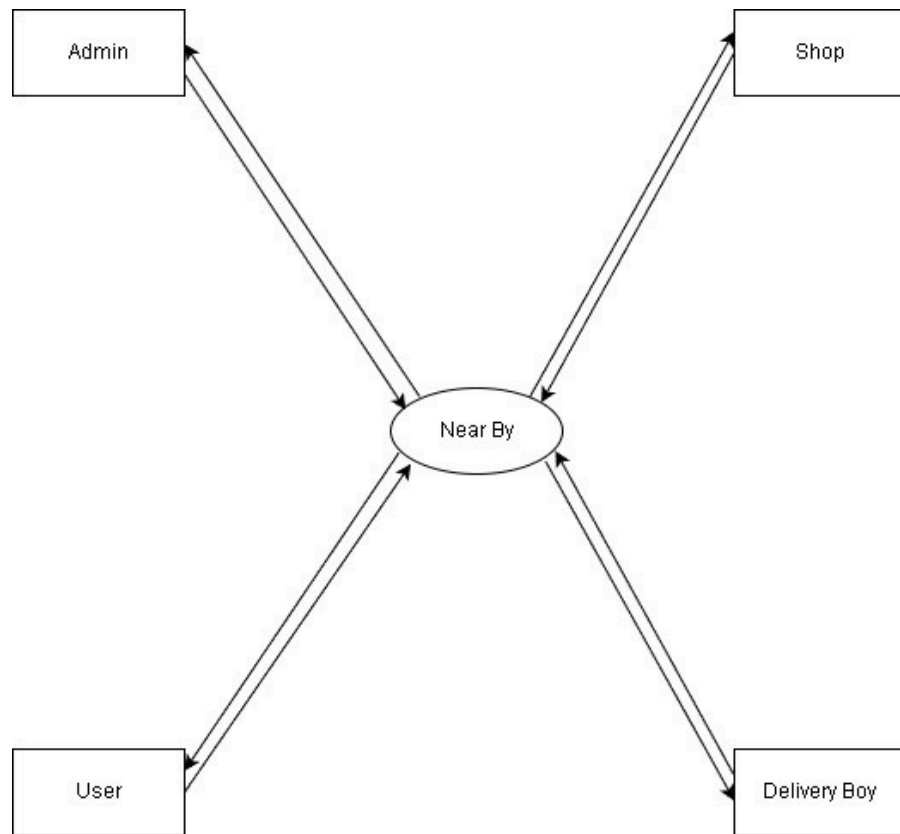


: Process

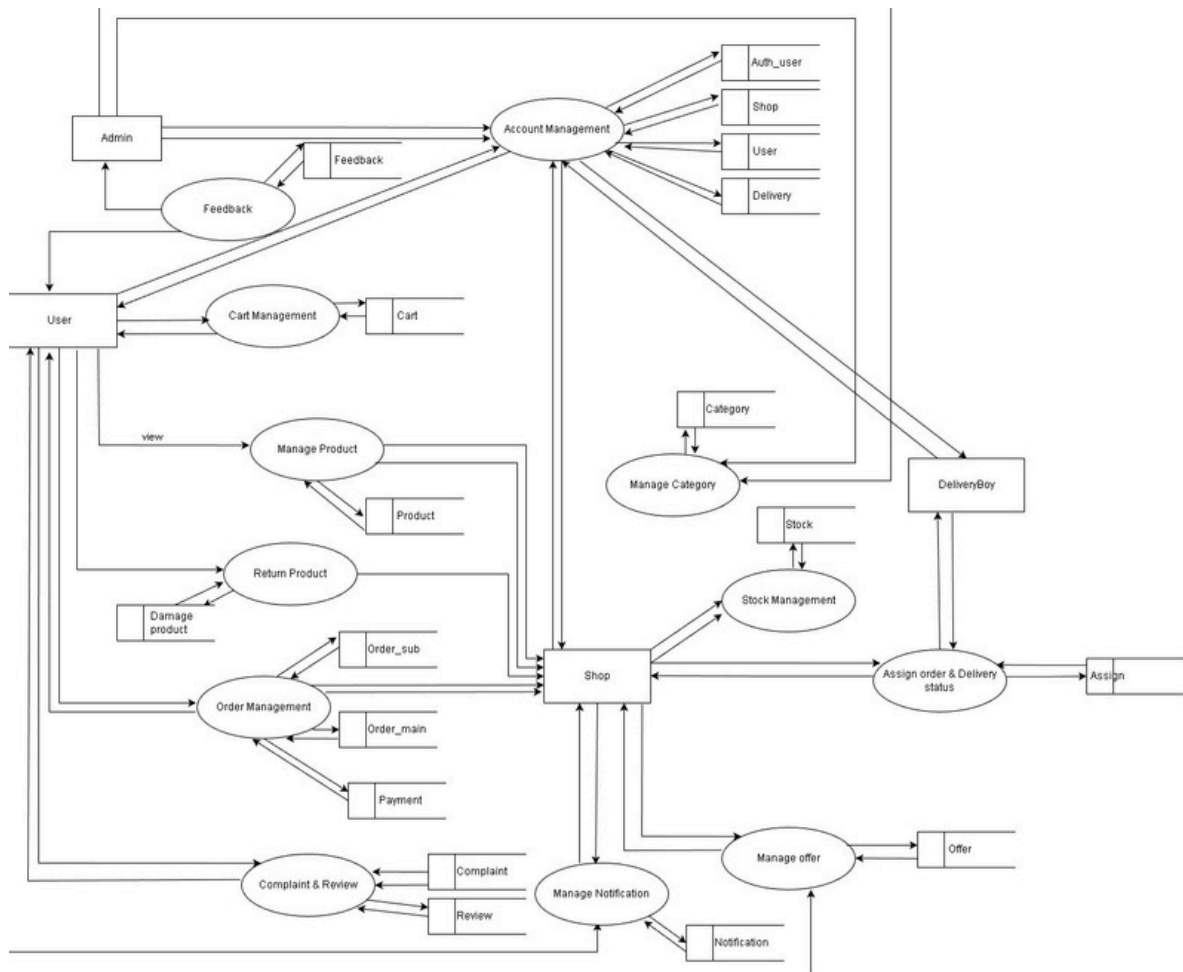


: Data Store

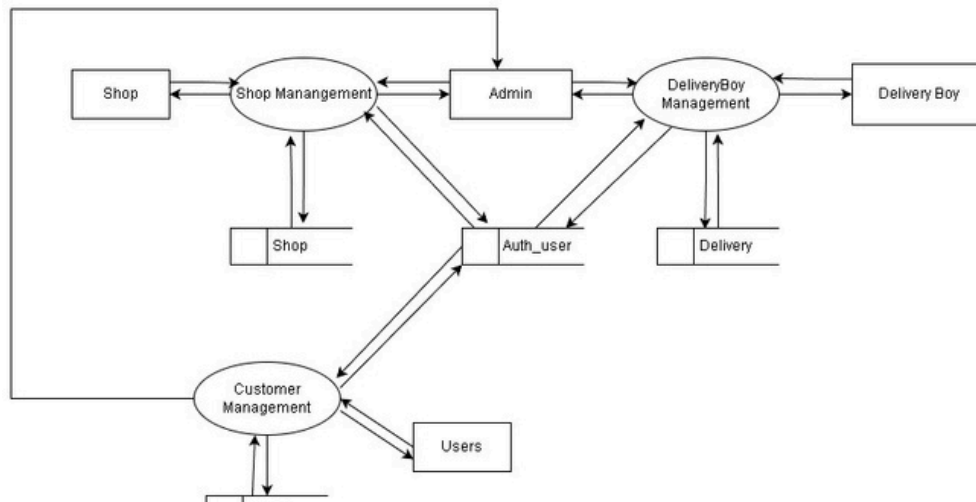
Level 0



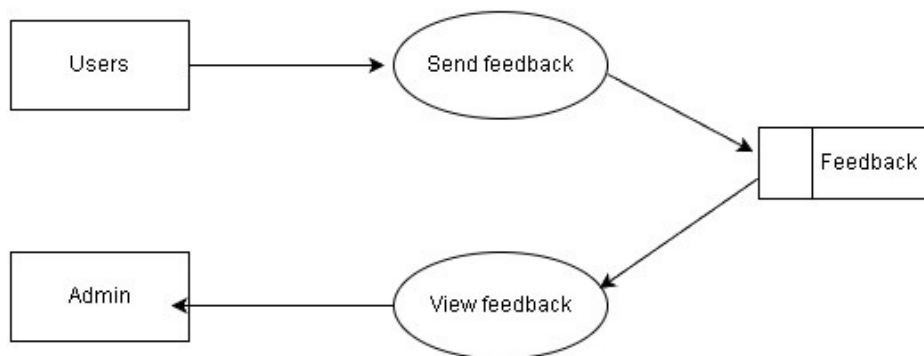
Level 1



Level 1.1



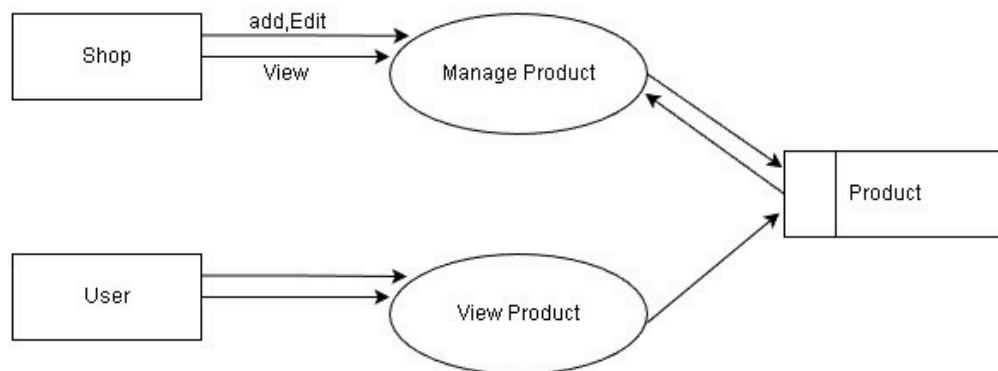
Level 1.2



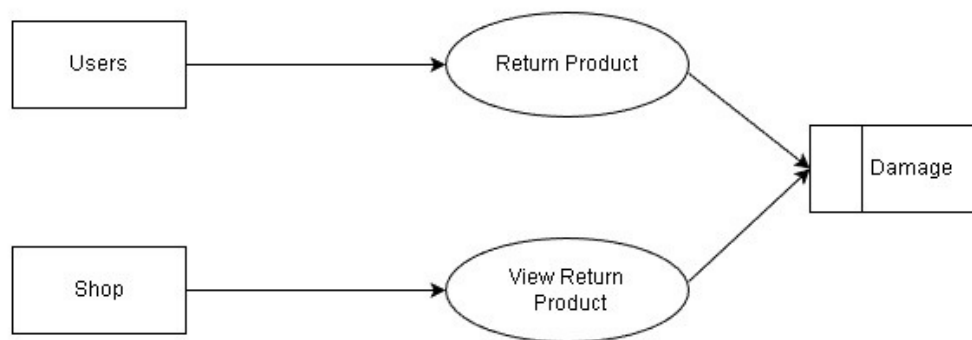
Level 1.3



Level 1.4



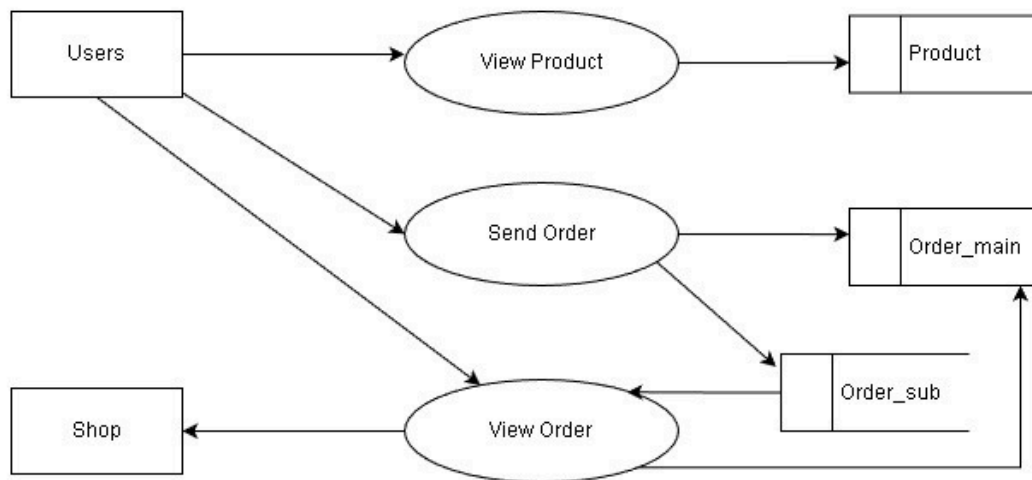
Level 1.5



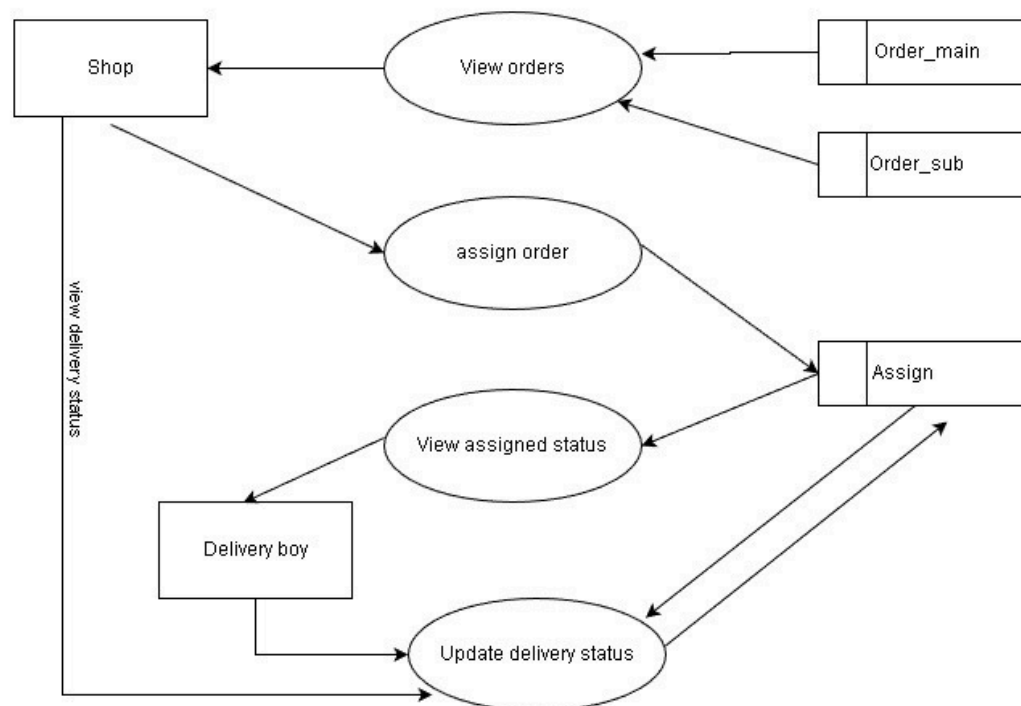
Level 1.6



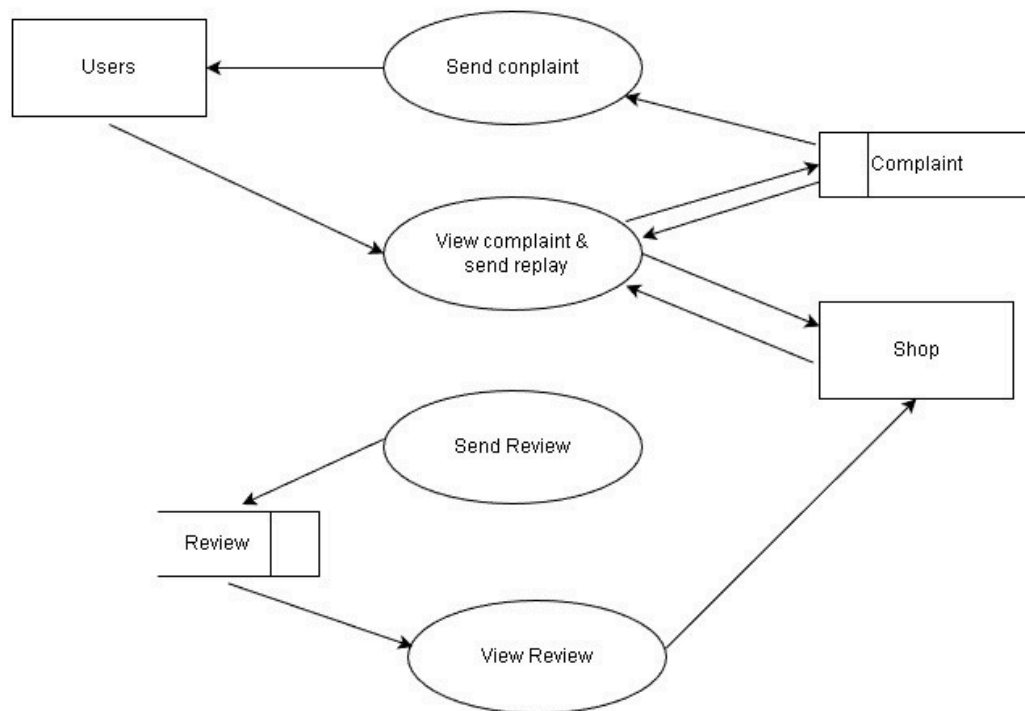
Level 1.7



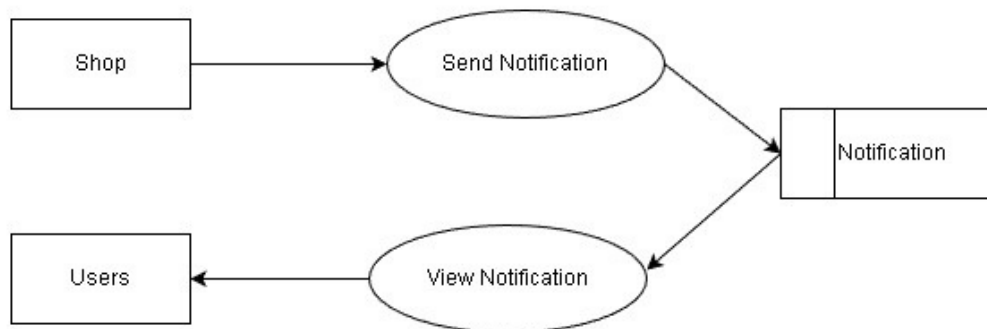
Level 1.8



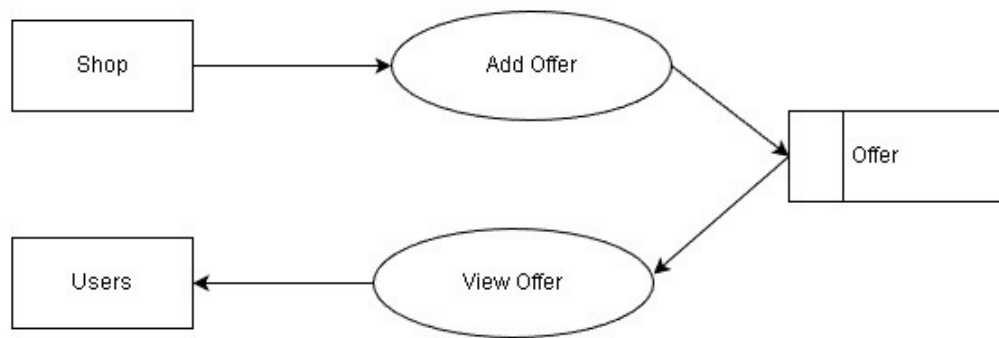
Level 1.9



Level 1.10



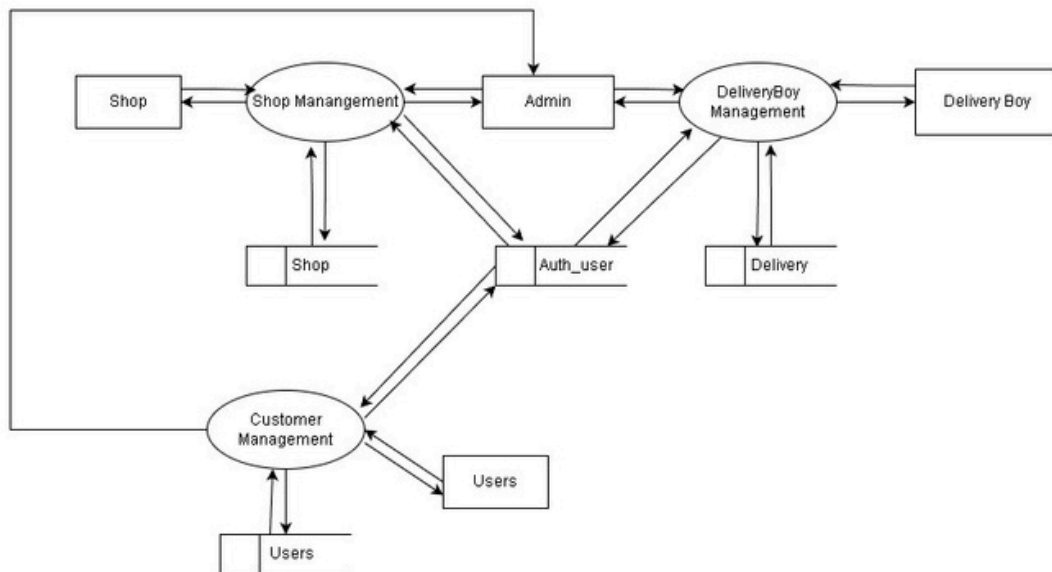
Level 1.11



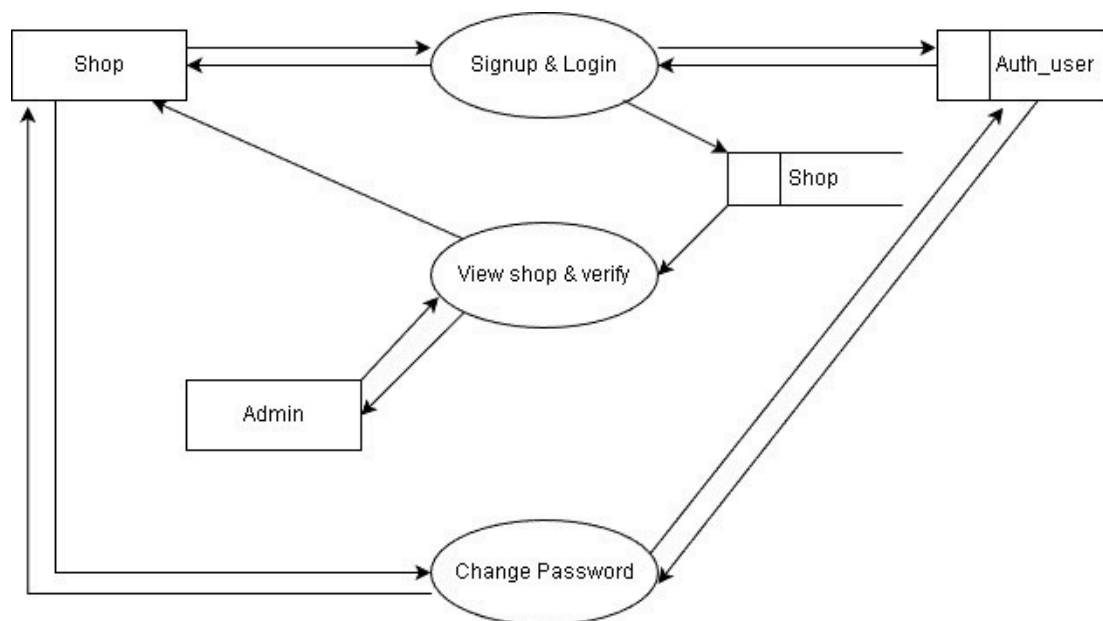
Level 1.12



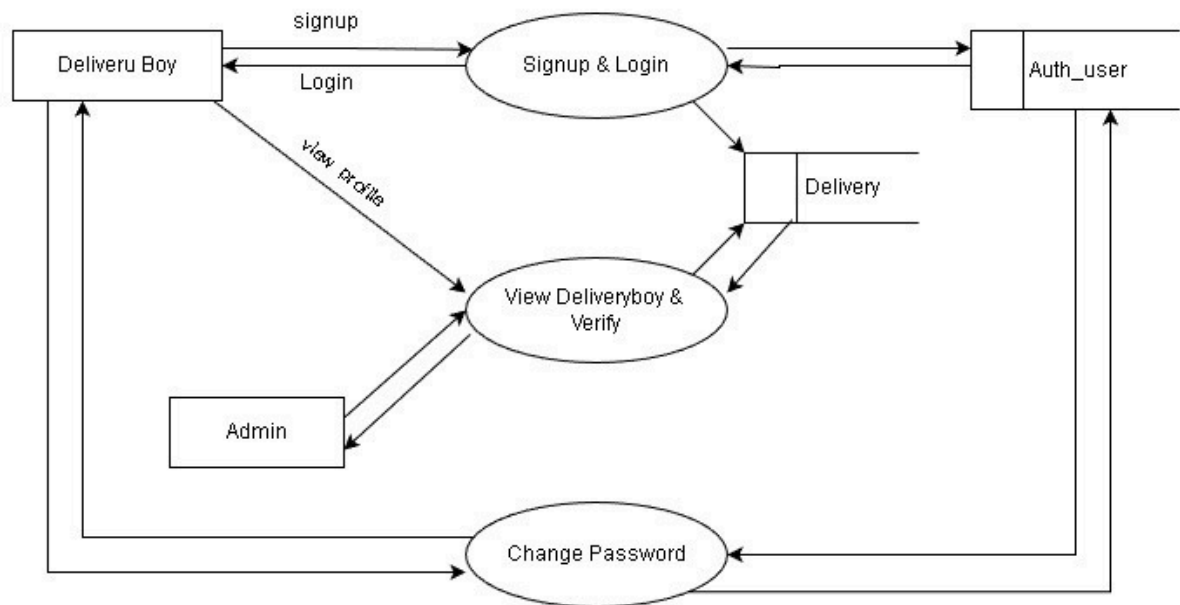
Level 2



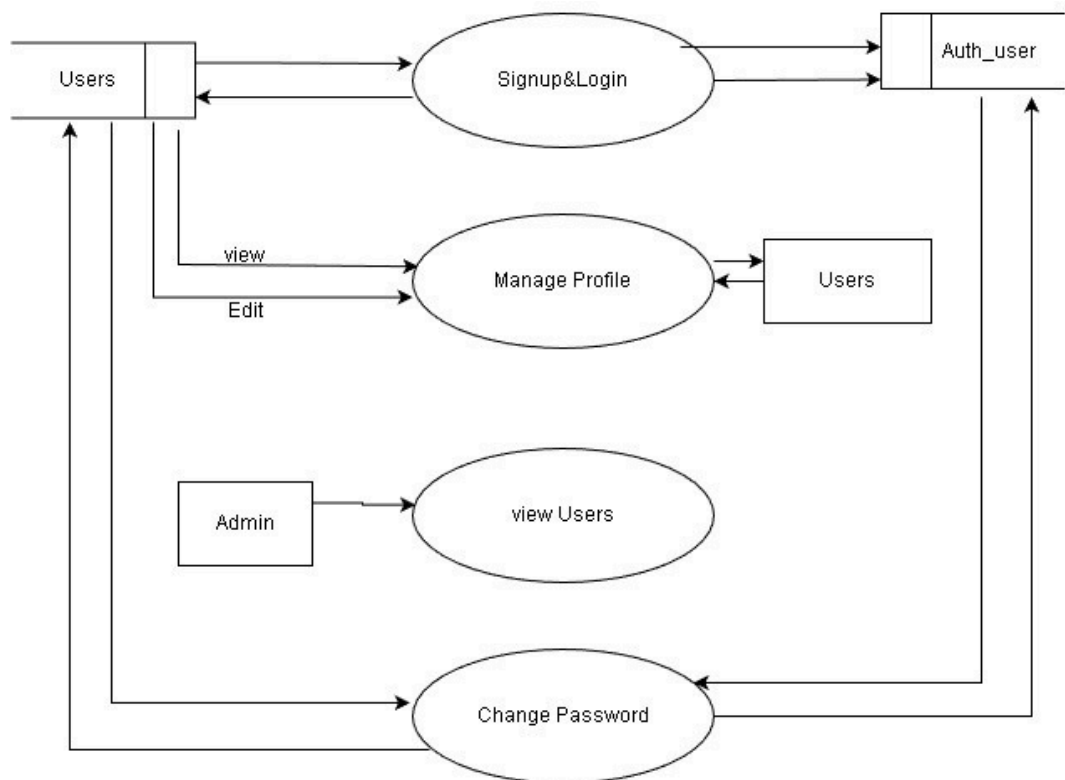
Level 2.1



Level 2.2



Level 2.3



6.6. Entity Relationship diagram

An entity relationship diagram is a data modelling technique that creates a graphical representation of the entities and the relationship between entities within an information system. An ER model is an abstract way to describe a database. Describing a database usually starts with a relational database, which stores data in tables. Some of the data in these tables point to data in the other tables. An entity may be defined as a thing which is recognized as being capable of an independent existent and this can be uniquely identified. An entity is an abstraction from the complicity of a domain. When we speak of an entity, we normally speak of some aspect of the real world which can be distinguished from other aspects of the real world.

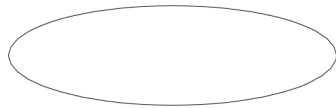
Three basic elements in ER model includes:

- ☐ Entities are the things about which we seek information
- ☐ Attributes are the data we collect about the entities
- ☐ Relationships provide the structure needed to draw information from multiple entities

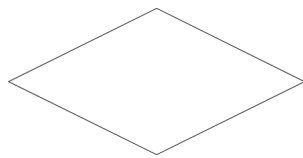
E-R Diagram Symbols



: Entity



: Attribute

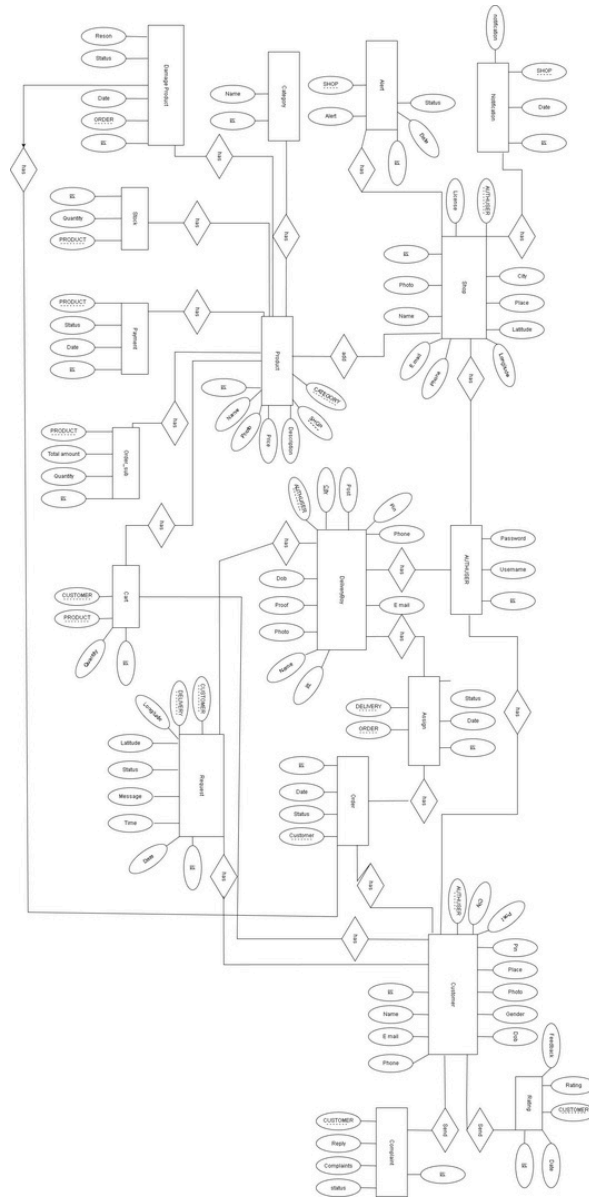


: Relation



: Connection

ER DIAGRAM



6.7. CLASS DIAGRAM

Aclass diagram is an essential part of the Unified Modelling Language (UML) used in software development to represent the static structure of a system. It provides a blueprint of the system's architecture by defining classes, their attributes, methods, and relationships. This diagram helps developers and stakeholders understand the object-oriented design and interaction between

different components. The primary purpose of a class diagram is to visualize the system's structure before implementation. It serves as a guide for software developers by defining the relationships between different entities. By clearly representing classes and their associations, it ensures a well-organized and scalable design that promotes modularity and reusability. It also plays a crucial role in documentation, making it easier for future developers to understand and maintain the system. A class diagram typically consists of multiple classes, each represented as a rectangle divided into three sections. The top section contains the class name, the middle section lists the attributes, and the bottom section includes methods or operations. The relationships between classes are depicted using different types of associations, including generalization (inheritance), aggregation, composition, and dependencies. In software development, class diagrams help implement object-oriented programming principles such as encapsulation, inheritance, and polymorphism. They define how objects interact within the system and establish clear dependencies, which aids in better code organization and maintainability. By illustrating how classes are interconnected, they provide a foundation for database schema design and backend system development. Class diagrams also enhance communication within development teams, allowing programmers, designers, and business analysts to align their understanding of the system. They serve as a reference document throughout the software lifecycle, facilitating testing, debugging, and future enhancements. Overall, a class diagram is a crucial tool in software engineering that ensures a structured and systematic approach to application development. It contributes to a clear and efficient system design, ultimately leading to a more robust and scalable software solution.

Visibility Modifiers in Class Diagrams:

- Public (+): Can be accessed by any class.

Private(-):Restricted to the class itself Protected(#):

Accessible by the class and its subclasses Package(~):

Accessible within the same package /module

Components of a Class diagram

Represented an rectangle divided into three section

Class Name (top section,bold or centered)

Attribute (middle section ,defining properties)

Method/Operation(button section, defining behaviors)

CLASS DIAGRAM

6.8. Activity Diagram

An activity diagram is a behavioral diagram in Unified Modelling Language (UML) that represents the dynamic aspects of a system by illustrating the flow of control or data between different activities. It visually describes the sequence of operations, decision points, and parallel processing within a system, making it an essential tool for understanding complex workflows and business processes. The primary purpose of an activity diagram is to model the logic of a process, whether it is a business workflow, a use case, or an algorithm. It helps in identifying the step-by-step execution of activities, including conditions and transitions that guide the flow of execution. This makes it useful for analyzing system behavior and optimizing processes before implementation. An activity diagram consists of several key elements, including activities, transitions, decision nodes, forks, joins, and swim lanes. Activities represent the tasks or operations performed, while transitions show the movement between activities. Decision nodes introduce conditional branches that lead to different execution paths based on specific conditions. Forks and joins enable parallel execution of multiple activities, ensuring synchronization where required. Swim lanes help categorize activities by grouping them based on responsible entities, clarifying accountability within a process. This diagram plays a significant role in software development by enhancing process visualization, improving communication among stakeholders, and providing a clear representation of system workflows. It aids developers in understanding process dependencies, identifying bottlenecks, and designing efficient workflows that enhance system performance. Activity diagrams are particularly useful in modelling business processes, system interactions, and workflow automation. They support various phases of software development, from requirement analysis to system design and implementation. By offering a high-level overview of a system's operation, they help in reducing ambiguities and ensuring a seamless transition from conceptualization to execution. Including an activity diagram in a project report

provides a structured representation of process flows, ensuring clarity and better comprehension for both technical and non-technical stakeholders. It contributes to effective project documentation, supports system validation, and facilitates future modifications and improvements.

Purpose Of Activity Diagrams

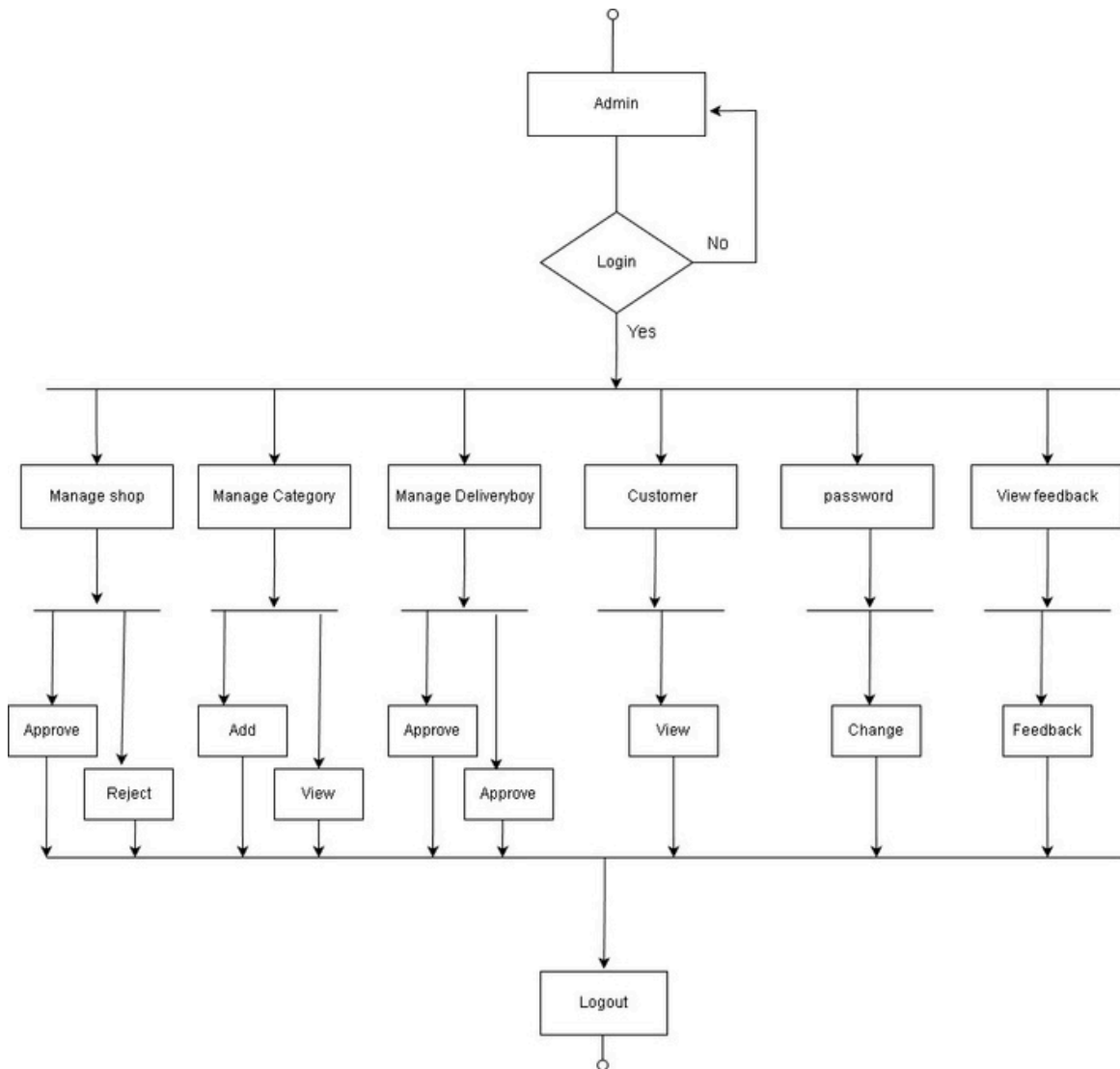
The purpose of an activity diagram is to visually represent the workflow or behavior of a system, showing the sequence of activities and the flow of control or data between them. It is used to model business processes, system operations, and functional workflows, making it easier to understand how different tasks are executed within a system. By providing a clear representation of the process logic, it helps in analyzing, designing, and improving system functionality. Activity diagrams play a crucial role in requirement analysis by identifying functional requirements and ensuring that all process flows are well-defined before implementation. They help developers and stakeholders understand how a system operates by depicting various execution paths, decision-making points, and interactions between different activities. This makes them useful for designing algorithms, workflows, and system processes. In software development, activity diagrams facilitate the design of efficient workflows by illustrating both sequential and parallel processing. By incorporating decision nodes, forks, and joins, they help in managing complex execution flows and multi-threaded processes. They also assist in identifying potential bottlenecks, improving system efficiency, and ensuring smooth execution of tasks. Beyond software design, activity diagrams serve as effective documentation tools, enhancing communication among developers, business analysts, and stakeholders. They provide a structured overview of system processes, making it easier to maintain, scale, and modify workflows as needed. Additionally, they support system testing by helping to identify test cases and potential failure points, ensuring a more reliable and optimized software solution.

Notations

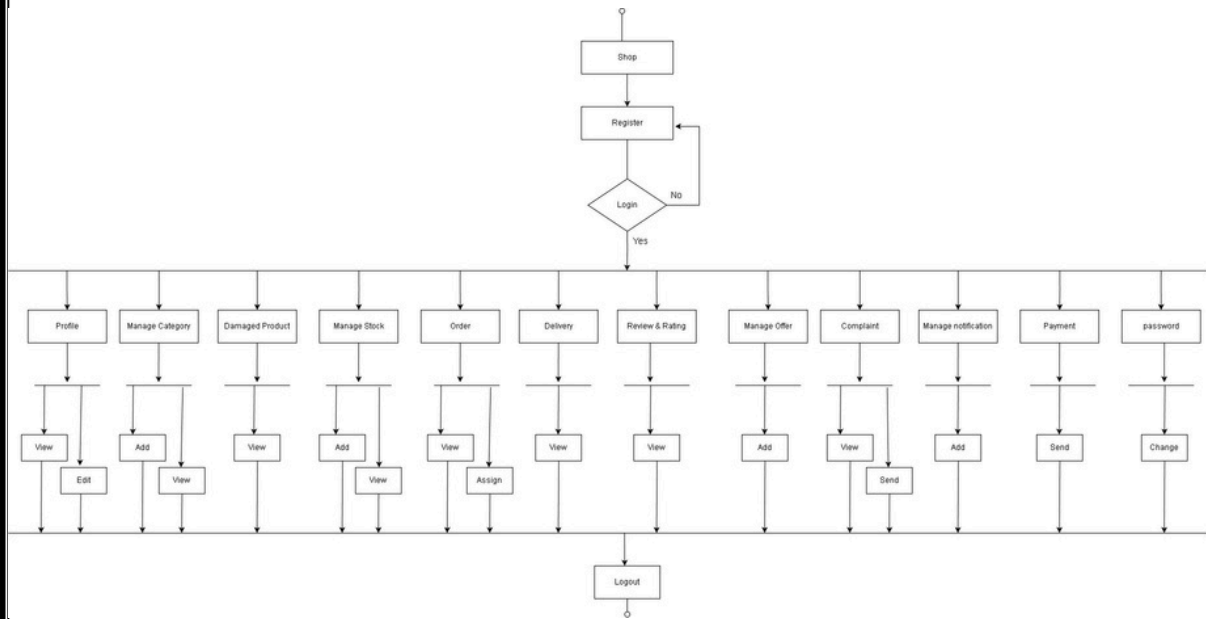
- Let's explore some common notations used in activity diagrams
- Initial State: Represented by a black filled circle, it marks the entry point into the system.
- Action or Activity State: Depicted as a rectangle with rounded corners, it represents an action or event.
- Action Flow or Control Flow: Arrows connecting activity states show transitions from one state to another

ACTIVITY DIAGRAM

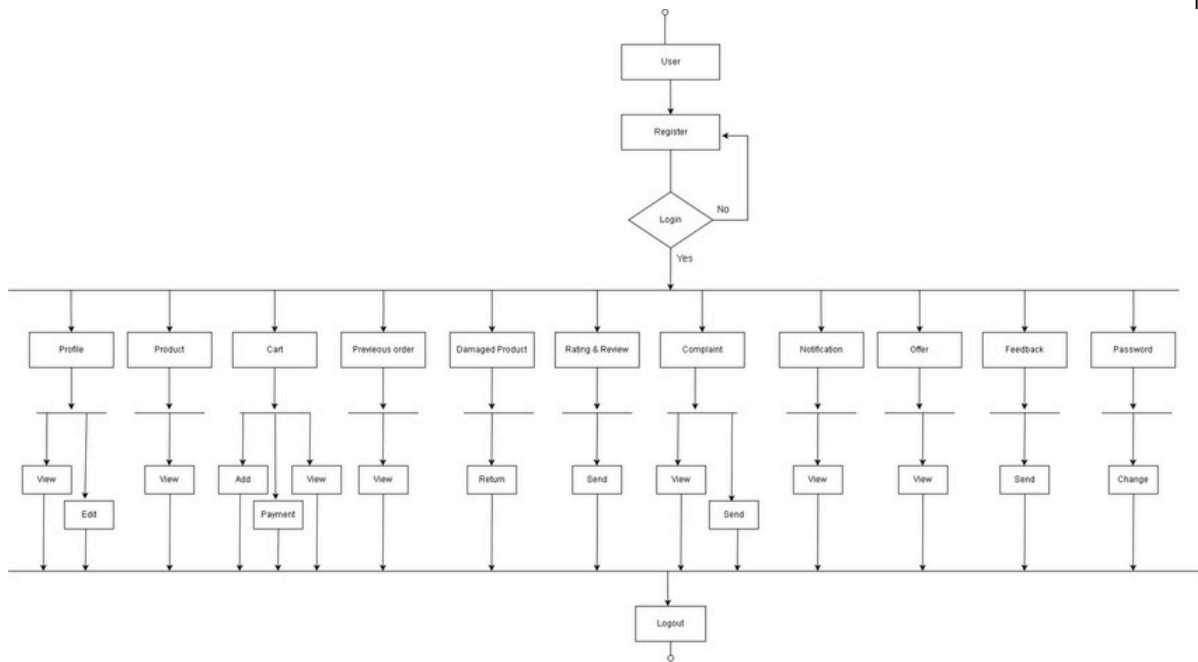
ADMIN



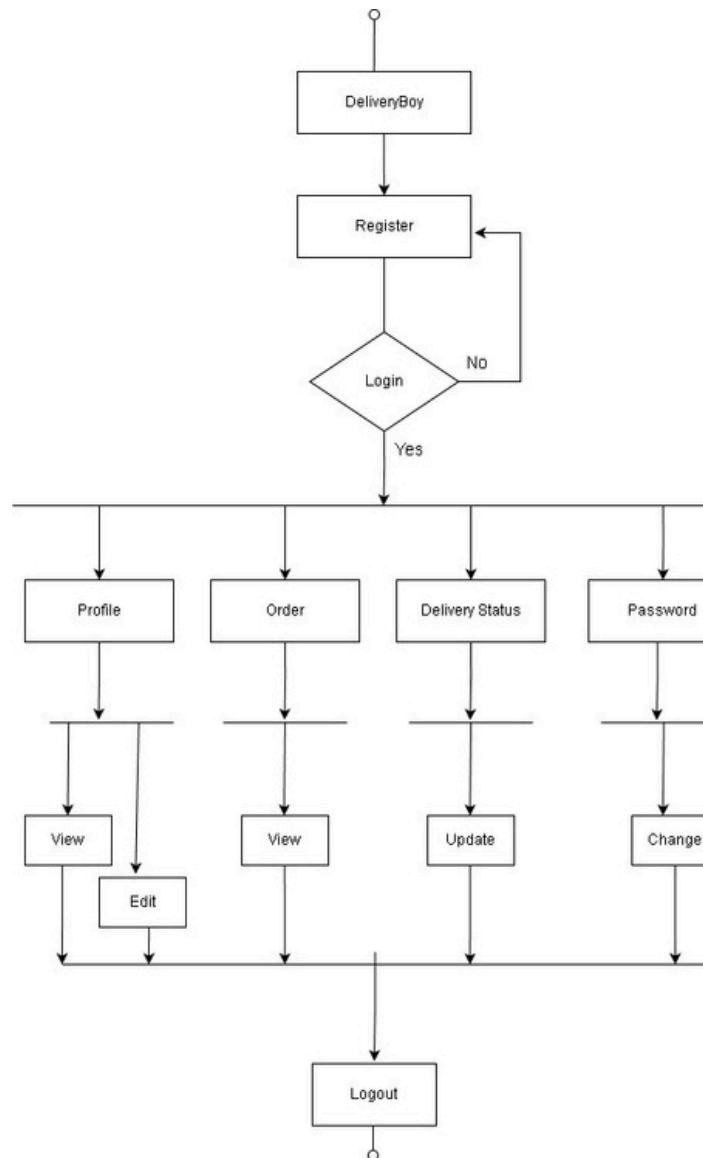
SHOP



USER



DELIVERY BOY



Database Design

A data base is a collection of inter related data stored with minimum redundancy to serve many quickly and efficiently. The general way is to make information accessing easy, quick, inexpensive and flexible for the user. In data base design several objectives are considered controlling redundancy, ease of learning and use data dependence, more information at low cost, accuracy and integrity are some of them. The design should be done in a way the information stored in the database can retrieved quickly whenever necessary. The general

theme behind a database is to handle information as an interfered whole. A database is a collection of interrelated data stored with minimum redundancy to serve users quickly and efficiently. Data base design runs parallel without application design. As we collect information about what is to be done, we will obviously collect information about data need to entered, stored messages and printed reports. The designing of database is done outmost care and security during the designing phase of the system. Special care was taken to develop minimum number of databases for the maximum efficiency of the system. The organization of the database aims to achieve the following objectives,

- Data independence
- Recovery from failures
- Privacy and security
- Performance
- Easy to learn in use

6.9. DATA DESIGN

Alert Table

FIELD NAME	DATA TYPE	CONSTRAINTS
id	bigint	Primary key, NN
date	date	NN
status	Varchar (100)	NN
alert	Varchar (100)	NN
SHOP_id	bigint	Foreign key ,NN

Assign Table

FIELD NAME	DATA TYPE	CONSTRAINTS
id	bigint	Primary key,NN
Date	date	NN
status	Varchar (100)	NN
DELIVERY_id	bigint	Foreign key ,NN
ORDER_id	bigint	Foreign key ,NN

Cart Table

FIELD NAME	DATA TYPE	CONSTRAINTS
id	bigint	Primary key ,NN
quantity	Varchar (100)	NN
CUSTOMER_id	bigint	Foreign key ,NN
PRODUCT_id	bigint	Foreign key ,NN

Notification Table

FIELD NAME	DATA TYPE	CONSTRAINTS
id	bigint	Primary key
date	date	Foreign key
notification	Varchar (200)	NN
SHOP_id	bigint	NN

Customer Table

FIELD NAME	DATA TYPE	CONSTRAINT
id	bigint Varchar	Primery key, NN
name	(100)	NN
email	Varchar (100)	NN
phone	Varchar (100)	NN
dob	date	NN
gender	Varchar (100)	NN
photo	Varchar (100)	NN
place	Varchar (100)	NN
pin	Varchar (100)	NN
post	Varchar (100)	NN
city	Varchar (100)	NN
AUTH_USER	int	Foreign key ,NN

Category Table

FIELD NAME	DATA TYPE	CONSRAINTS
id	bigint	Primary key ,NN
category	Varchar (100)	NN

Complain Table

FIELD NAME	DATA TYPE	CONSRANTS
id	bigint	Primary key ,NN
date	date	NN
status	Varchar (100)	NN
complaint	Varchar (100)	NN
reply	Varchar (100)	NN
CUSTOMER_id	bigint	Foreign key ,NN
PRODUCT_id	bigint	Foreign key ,NN

Damaged product Table

FIELD NAME	DATA TYPE	CONSRANTS
id	bigint	Primary key ,NN
date	date	NN
status	Varchar (100)	NN
reason	Varchar (200)	NN
PRODUCT_id	bigint	Foreign key ,NN
CUSTOMER_id	bigint	Foreign key ,NN

Delivery boy Table

FIELD NAME	DATA TYPE	CONSRAINTS
id	bigint	Primary key ,NN
name	Varchar (100)	NN
email	Varchar (100)	NN
phone	Varchar (100)	NN
Dob	date	NN
proof	Varchar (100)	NN
photo	Varchar (100)	NN
place	Varchar (100)	NN
pincode	Varchar (20)	NN
post	Varchar (100)	NN
City	Varchar (100)	NN
AUTHUSER_id	int	Foreign key,NN
status	Varchar (100)	NN

Offer Table

FIELD NAME	DATATYPE	CONSTRAINT
id	bigint	Primary key ,NN
From_date	date	NN
To_date	date	NN
From_time	Time (6)	NN
To_time	Time (6)	NN
Photo	Varchar (200)	NN
Description	Varchar (200)	NN
Offer_name	Varchar (200)	NN
PRODUCT_id	Bigint	Foreign key ,NN

Order Table

FIELD NAME	DATA	CONSTRAINT
Id	TYPE bigint	Primary key,NN
date	date	NN
status	Varchar (100)	NN Foreign key
CUSTOMER_id	bigint	,NN Foreign key
SHOP_id	bigint	,NN

Order sub Table

FIELD NAME	DATA TYPE	CONSTRAINT
id	bigint	Primary key,NN
quantity	Varchar (100)	NN
total_amount	Varchar (100)	NN Foreign key
PRODUCT_id	bigint bigint	,NN Foreign key
ORDER_id		,NN

Payment Table

FIELD NAME	DATA TYPE	CONSTRAINT
Id	bigint	Primary key,NN
amount	Varchar (100)	NN
date	date	NN
status	Varchar (100)	NN
PRODUCT_id	bigint	Foreign key ,NN

Product Table

FIELD NAME	DATA TYPE	CONSTRAINT
Id	bigint	Primary key,NN
name	Varchar (100)	NN
photo	Varchar (100)	NN
price	Varchar (50)	NN
description	Varchar (200)	NN
CATEGORY_id	bigint	Foreign key ,NN
SHOP_id	bigint	Foreign key ,NN

Rating Table

FIELD NAME	DATA TYPE	CONSTRAINT
Id	bigint	Primary key,NN
date	date	NN
feedback	Varchar (200)	NN
Rating	Varchar (200)	NN Foreign key
CUSTOMER_id	bigint bigint	,NN Foreign key
SHOP_id		,NN

Request Table

FIELD NAME	DATA TYPE	CONSTRAINT
Id	bigint	Primary key,NN
date	date	NN
Time	time	NN
Message	Varchar (200)	NN
status	Varchar (100)	NN
latitude	Varchar (100)	NN
longitude	Varchar (100)	NN
CUSTOMER_id	bigint	Foreign key ,NN
DELIVERY_id	bigint	Foreign key ,NN

Shop Table

FIELD NAME	DATA TYPE	CONSTRAINT
Id	bigint	Primary key,NN
photo	Varchar (700)	NN
name	Varchar (50)	NN
email	Varchar (100)	NN
phone	Varchar (20)	NN
place	Varchar (50)	NN
city	Varchar (50)	NN
latitude	Varchar (50)	NN
longitude	Varchar (50)	NN
license	Varchar (100)	NN
AUTHUSER_id	int	Foreign key ,NN
status	Varchar (100)	NN

Stock Table

FIELD NAME	DATA TYPE	CONSTRAINT
Id	bigint	Primary key,NN
quantity	Varchar (100)	NN
PRODUCT_id	bigint	Foreign key ,NN

SYSTEM TESTING & IMPLEMENTATION

7.1. System testing

System testing is a level of software testing where a complete and integrated software is tested. The purpose of this test is to evaluate the system's compliance with the specified requirements. It makes a logical assumption that if all the parts of the system are correct, the goal will be successfully achieved. The candidate system is subject to a variety of tests: online response, volume, stress, recovery and security and usability test. It is the process of exercising or evaluating a system by manual or automatic means to verify that it satisfies the specified requirements or to identify the difference between expected and actual results. The testing activities are aimed at convincing the customer through demonstration and actual use that the software is a solution to the original problem and that both the product and the process that created it are of high quality. It is also used to find and eliminate any residual errors from previous stages and the operational reliability of the system.

Preparation of Testing Data

Software testing is a crucial element of software quality assurance and represents the ultimate review of specification, design and coding. Testing represents an interesting anomaly for the software. During earlier definition and development phases, it was attempted to build software from abstract concepts to tangible implementation. The testing responsible for ensuring that the product that has been built performs the way that the detailed design documentation specifies.

Methods of Testing

Testing is the process of finding bugs in a program. It helps to improve the quality of the software. It has to be done thoroughly and with the help of

specialist testers. System testing is a process of checking whether the developed system is working according to the objectives and requirements. The used testing methods are:

- Unit Testing
- Integration Testing
- Validation Testing
- Output Testing
- User Acceptance Testing

Unit Testing

The first level of testing is called as unit testing. Here the different modules are tested and the specification produced during design for the modules. Unit testing is essential for verification of the goal and to test the internal logic of the modules. Unit testing is conducted to different modules of the project. Errors were noted down and corrected down immediately and the program clarity was increased. The testing was carried out during the programming stage itself. In this step each modules found to be working satisfactory as regard to expected from the module.

Integration Testing

The second level of testing includes integration testing. It is a systematic testing of constructing structure. At the same time tests are conducted to uncover errors with the interface. It need not to be the case, that software whose modules when run individually showing results will also show perfect results when run as a whole. The individual modules are tested again and the results are verified. The goal is to see if the modules integrated between the modules. This testing activity can be considered as testing the design and emphasizes on testing modules interaction.

Validation Testing

The next level of testing is validation testing. Here the entire software is tested. There Ference document for this process is the requirement and the goal is to see if the software meets its requirements. The requirement document reflects and determines whether the software functions as the user expected. At culmination of integration testing, software is completely assembled as a package and corrected and a final series of software test validation test begins. The proposed system under construction has been tested by using validation testing and found to be working satisfactory. Data validation checking is done to see whether the corresponding entries made in different tables are done correctly. Proper validation checks are done in case of insertion and updating of tables, in order to see that no duplication of data has occurred. If any such case arises proper warning message will be displayed.

Output Testing

The output of the software should be accepted able to the system user. The output of requirement is defined during the system analysis. Testing of this often ware system is done against the output and the output testing was completed with success.

User Acceptance Testing

An acceptance test has the objective of selling the user on the validity and reliability of the system. It verifies that the system procedures operate to system specification and the integrity of the vital data is maintained.

7.2. System implementation

System implementation is the final phase i.e., putting the utility into action.

Implementation is the state in the project where theoretical design turned into working system. Implementation involves the conversion of a basic application to computer placement with a computer system. It is the process of converting to a new or revised system design into an operation alone. During the design phase, the products structure, its underlying data structures, the general algorithms and the interfaces and control/data linkages needed to support communication among the various sub structures were established. Implementation process is simply a translation of the design abstraction into the physical realization, using the language of the target architecture. Implementation includes all those activities that take place to convert from the old system to the new. The new system may be totally new replacing an existing manual or automated system, or it may be major modification to an existing system. In either case, proper implementation is essential to provide a reliable system to meet organizational requirements. There are three types of implementations:

- Implementation of a computer system or place manual system.
- Implementation of a new computer system to replace an existing one.
- Implementation of a modified application to replace an existing one, using the same computer.

The common approaches for implementation are:

Parallel Conversion

In parallel conversion the existing system and new system operates simultaneously until the project team is confident that the new system is working properly. The outputs from the old system continue to be distributed until the new system has proved satisfactorily parallel conversion is a costly method because of the number of duplications involved.

Direct Conversion

Under direct conversion method the old system is discontinued together and the new system becomes operational immediately. A greater risk is associated with direct conversion is no backup in the case of system fails.

Pilot Conversion

Pilot conversion would involve the changing over of the part of the system either in parallel or directly. Use of the variation of the two main methods is possible when part of the system can be treated as a separate entity.

User Training

After the system is implemented successfully, training of the user is one of the most important sub tasks of the developer. For this purpose user manuals are prepared and handed over to the user to operate the developed system. Thus the users are trained to operate the developed system. In order to put new application system into use, the following activities were taken care of:

- Preparation of user and system documentation.
- Conducting user training with demo and hand son.
- Test run for some period to ensure smooth switching over the system.

The major implementation procedures are:

- Test plans
- Training
- Conversion

Test Plans

The implementation of a computer-based system requires that the test data be prepared and the system and its elements be tested in a structured manner.

Training

The purpose of training is to ensure that all the personal who are to be associated with the computer-based business system possesses the necessary knowledge skills. As the system provides user friendliness only basic training is needed.

Conversion

It is the process of performing all of the operations that results directly in the turnover of the new system to the user. Conversion has two parts: The creation of a conversion plan at the start of the development phase and the implementation of the plan throughout the development phase. The creation of a system changes over plan at the end of the development phase and the implementation of the plan at the beginning of the operation phase.

CODING

When large volumes of data are being handled, it is important that items must be identified, stored or selected easily and quickly. To accomplish this each data item must have a unique identification and must be related to other item of data of the same type. Code can provide briefs identification of data item and replace longer description that would be more specific and manipulated. A code has a group of characters used to identify an item of data

CODE PLAN

The code plans identifies particular characteristic that need to be continue within The code. Whatever be the nature od code they must be:

- Exanpadable this code must provide space for additional entries that maybe required
- Precise This code identifies as specific form

- Concise This code must brief and yet adequately describe this item
- Convenient this code must easily encode and decode
- Meaningful This code must be using familiar the people dealing with it

SYSTEM MAINTAINANCE

8.1. Corrective Maintenance

Corrective maintenance refers to changes made to repair defects in the design, coding, or implementation of the system. Corrective maintenance is often needed for repairing processing or performance failures or making change because of previously uncorrected problems or false assumptions. Most corrective maintenance problems surface soon after the installation. When corrective maintenance problems surface, they are typically urgent and need to be resolved to curtail possible interruptions in normal business activities.

8.2. Adaptive Maintenance

Adaptive maintenance involves making changes to an information system to evolve its functionality or to migrate it to a different operating environment. Adaptive Maintenance is usually less urgent than corrective maintenance because business and technical changes typically occur some period of time.

8.3. Perfective Maintenance

Perfective maintenance involves making enhancements to improve processing performance, interface usability, or to add desired, but not necessarily required, system features. Many system professionals feel that perfective maintenance is not really the maintenance but new development.

8.4. Preventive Maintenance

Preventive maintenance is the only maintenance activity which is carried out without formal maintenance request from the user. When a software company or maintenance agency realizes that the methodologies used in a program have

become obsolete, it may decide to develop or modify parts of the program, which do not confirm to the current trends. Of the steps, more time and money is spending on perfective than on corrective and adaptive maintenance together.

FUTURE ENHANCEMENT

9. FUTURE ENHANCEMENT

The future vision for the Seat Sharing app is both ambitious and forwardthinking, aiming to revolutionize the ride-sharing experience. Here are the key features and technologies that will drive its evolution:

1. AI-Based Product Recommendation

- Provide personalized product suggestions to users based on previous purchases and browsing history.
- Help customers quickly find relevant products from nearby shops.

2. Smart Shop Ranking System

- Rank nearby shops based on factors such as ratings, availability, pricing, and user preferences.
- Improve visibility for high-performing local shops.

3. Advanced Order Management

- Introduce automated order prioritization for urgent or essential products such as medicines and groceries.
- Improve coordination between shops and delivery agents for faster processing.

4. Multi-Language Support

- Allow users to access the application in multiple regional languages.
- Increase accessibility for a wider range of users, especially local customers.

5. Enhanced Security and Authentication

- Implement advanced authentication methods such as OTP-based login and biometric verification.
- Improve data protection and user trust within the system.

6. Offers, Rewards, and Loyalty Programs

- Introduce reward points, discounts, and cashback offers for frequent users.
- Encourage customers to support local shops through loyalty benefits.

Overall, these future enhancements will help NearBuy evolve into a more powerful local commerce platform, improving user satisfaction, strengthening local businesses, and ensuring long-term scalability and efficiency

CONCLUSION

10. Conclusion

In conclusion, the NearBuy application presented in this project provides an efficient and user-friendly solution for local shopping and delivery needs. By connecting customers with nearby shops through a single mobile platform, the system simplifies the process of finding products, placing orders, and choosing between home delivery or direct shop pickup. This approach significantly reduces the time and effort required for traditional shopping while improving overall convenience.

The system also plays an important role in supporting local businesses by offering them a digital platform to manage products, handle orders, and reach nearby customers without investing in complex infrastructure. By integrating delivery agents into the system, NearBuy ensures reliable order fulfillment while creating flexible employment opportunities. This collaboration between users, shops, and delivery personnel enhances the efficiency of local commerce.

Overall, the NearBuy application promotes smarter and faster local shopping while strengthening the connection between customers and local businesses. With the possibility of future enhancements such as improved recommendations, advanced security, and loyalty programs, the system has strong potential to evolve into a scalable and sustainable local commerce platform that benefits both users and small businesses.

BIBLIOGRAPHY

11. BIBLIOGRAPHY

Books

- Sommerville, I. (2016). Software engineering (10th ed.). Pearson Education.
- Napoli, M. L. (2019). Beginning Flutter: A hands-on guide to app development. Wiley.
- Bailey, T. (2020). Flutter for beginners: A step-by-step guide to building apps. Packet Publishing.

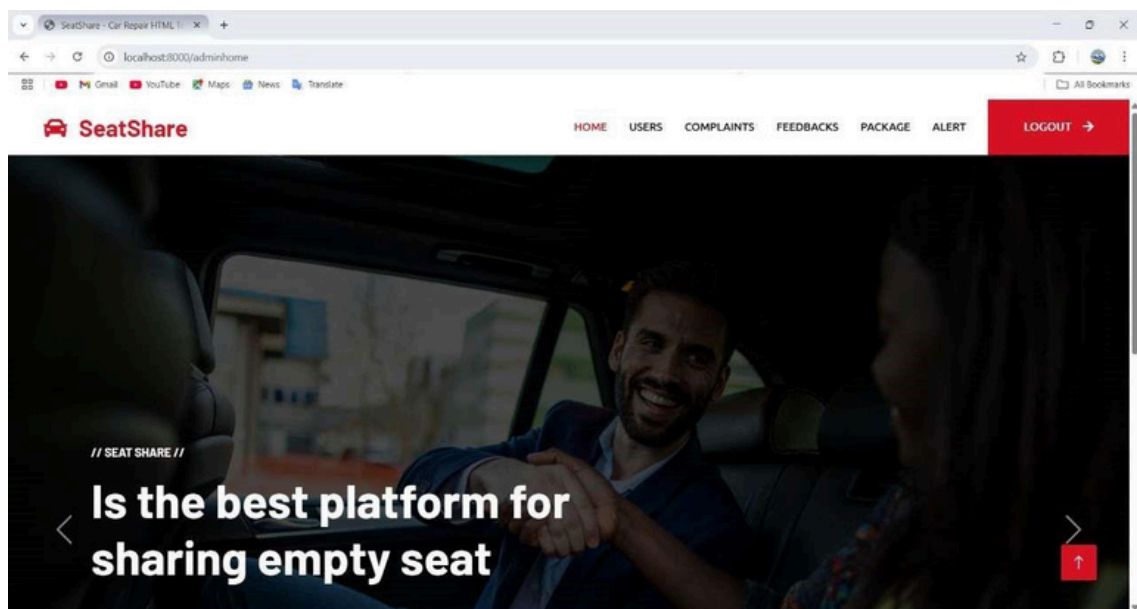
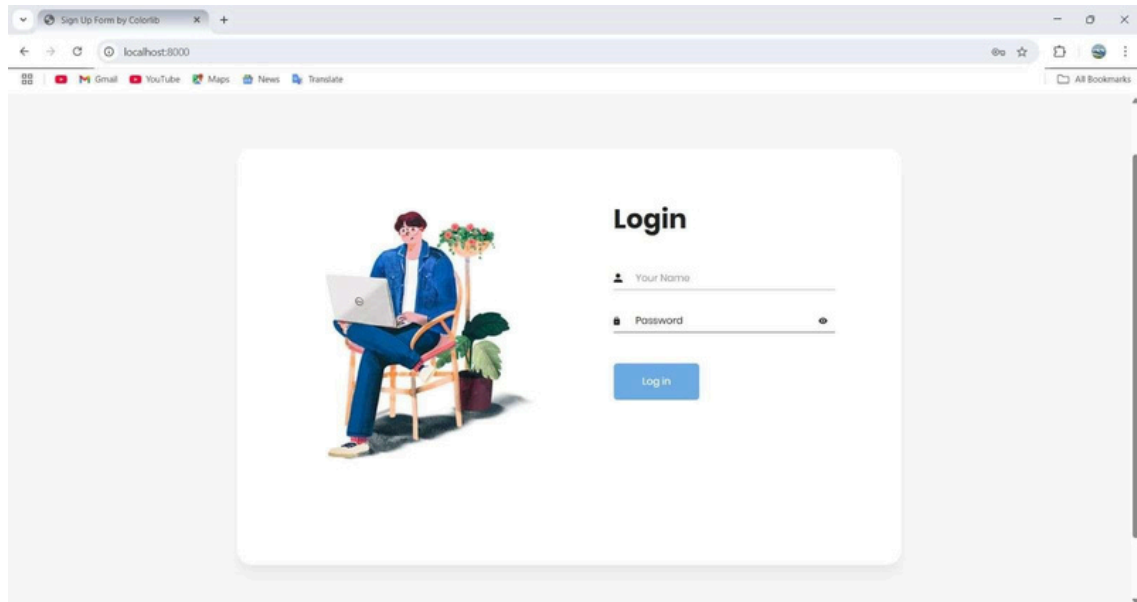
Web Resources

- MDN Web Docs. (n.d.). HTML, CSS, and JavaScript documentation. Retrieved from <https://developer.mozilla.org>
- Bootstrap. (n.d.). Bootstrap documentation. Retrieved from <https://getbootstrap.com>
- Google Developers. (n.d.). Dart programming language documentation. Retrieved from <https://dart.dev>
- Django Project. (n.d.). Django documentation. Retrieved from <https://www.djangoproject.com>
- Oracle. (n.d.). MySQL documentation. Retrieved from <https://www.mysql.com>
- JetBrains. (n.d.). PyCharm IDE documentation. Retrieved from <https://www.jetbrains.com/pycharm>
- Google. (n.d.). Android Studio documentation. Retrieved from <https://developer.android.com/studio>
- Flutter. (n.d.). Flutter documentation. Retrieved from <https://flutter.dev>

APPENDIX

12.1. Sample Screenshots

ADMIN



SeatShare - Car Repair HTML

localhost:8000/view_user#a

HOME USERS COMPLAINTS FEEDBACKS PACKAGE ALERT LOGOUT

SEARCH

sl	name	email	phone	place	gender	age	adhaar_no	Licence No	manage
1	sabith	muhammedsabithpp313@gmail.com	8606774116	Kozhikode, Kerala, India	Male	22	336878437741	KL57482356	BLOCK
2	nihal	nihal@gmail.com	9645097178	Kozhikode, Kerala, India	Male	28	336878435522	No License	BLOCK
3	Muhammed	muhammed@gmail.com	9685740231	Kozhikode, Kerala, India	Male	23	968574007808	No License	BLOCK
4	koya	koya@gmail.com	9645785286	Kozhikode, Kerala, India	Male	28	852074109630	No License	BLOCK

localhost:8000/adminhome

SeatShare - Car Repair HTML

localhost:8000/view_alert_details#a

HOME USERS COMPLAINTS FEEDBACKS PACKAGE ALERT LOGOUT

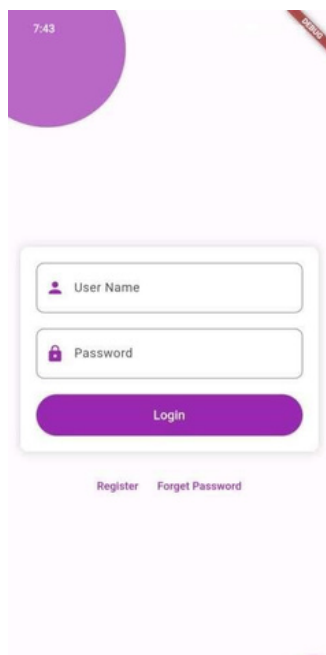
VIEW LOCATION

Sl	User	User Phone No	Date	Alert	Driver	Licence No	Driver Phone No	Location
1	nihal	9645097178	March 2, 2025	Fake ID Person	sabith	KL57482356	8606774116	VIEW LOCATION
2	nihal	9645097178	March 7, 2025	Fake ID Person	sabith	KL57482356	8606774116	VIEW LOCATION

© SEAT SHARE. All Right Reserved.

Home

USER



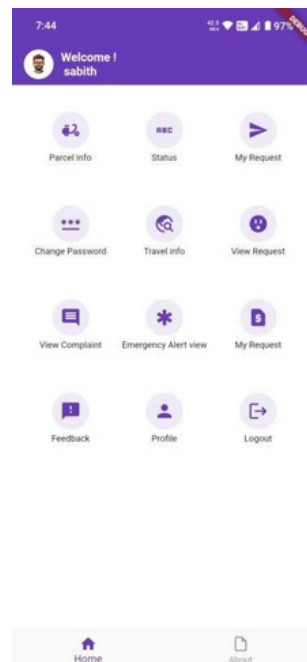
7:43

User Name

Password

Login

Register Forget Password



7:44

Welcome !
sabith

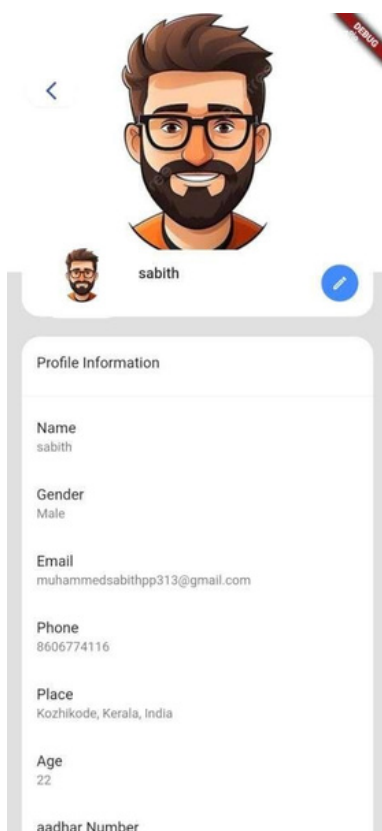
Parcel info Status My Request

Change Password Travel info View Request

View Complaint Emergency Alert view My Request

Feedback Profile Logout

Home About



<

sabith

Profile Information

Name
sabith

Gender
Male

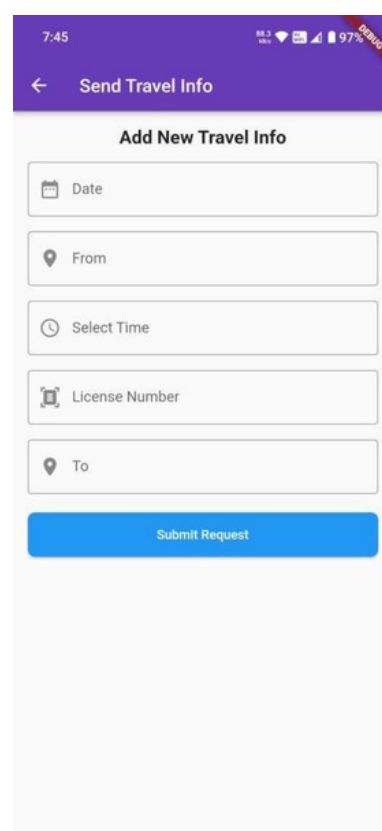
Email
muhammedsabithpp313@gmail.com

Phone
8606774116

Place
Kozhikode, Kerala, India

Age
22

aadhar Number



7:45

Send Travel Info

Add New Travel Info

Date

From

Select Time

License Number

To

Submit Request

12.2. Sample Codes

```

import          'dart:convert';
'package:flutter/material.dart';
'package:fluttertoast/fluttertoast.dart';
'package:seatshare/profile.dart';
'package:seatshare/send%20alert.dart';
'package:seatshare/send_request.dart';
'package:seatshare/share_parcel_details.dart';
'package:seatshare/share_request_driver.dart';
'package:seatshare/user_view_emergency_alert.dart';
import 'package:seatshare/view_complaint.dart';
'package:seatshare/view_others_travel_info.dart';
'package:seatshare/view_own_parcel.dart';
'package:seatshare/view_own_request.dart';
'package:seatshare/view_send_parcel.dart';
'package:seatshare/view_send_request.dart';
'package:http/http.dart' as http;
'package:shared_preferences/shared_preferences.dart';
import 'SendFeedback.dart';

import 'about_app.dart';
import
'change_password.dart';
//import 'login.dart';
import 'mytraininfo.dart';
import 'newlogin.dart';

void main() {
runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
return MaterialApp(
debugShowCheckedModeBanner: false,
home: HomeScreen(),
  );
}
}

class HomeScreen extends StatefulWidget {
  @override
  _HomeScreenState createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> { int
_selectedIndex = 0; // Tracks selected tab in Bottom
Navigation String name = 'name';

```



```

String gender = 'gender';
String email = 'email';
String phone = 'phone';
String place = 'place';
String age = 'age';
String aadhar_no = 'aadhar_no';
String liceNo = 'liceNo';
String image = 'image';

@Override
void
initState() {
super.initState();
senddata();

}

void senddata() async {
SharedPreferences sh = await SharedPreferences.getInstance();
String url = sh.getString('url').toString();
String lid = sh.getString('lid').toString();

```

```

    final urls = Uri.parse(url +
"view_profile");    try {    final response
= await http.post(urls, body: {
    'lid': lid,
    });
    if (response.statusCode == 200) {
        String status =
jsonDecode(response.body)['status'];    if (status
== 'ok') {
        setState(() {    name = jsonDecode(response.body)['name'].toString();
gender = jsonDecode(response.body)['gender'].toString();    email =
jsonDecode(response.body)['email'].toString();    place =
jsonDecode(response.body)['place'].toString();    age =
jsonDecode(response.body)['age'].toString();    phone =
jsonDecode(response.body)['phone'].toString(); aadhar_no =
jsonDecode(response.body)['aadhar_no'].toString(); liceNo =
jsonDecode(response.body)['liceNo'].toString();    image =
sh.getString('imgurl').toString() +jsonDecode(response.body)['image'];
        });
        }else {
Fluttertoast.showToast(msg: 'Not Found');
        }
        }else {
Fluttertoast.showToast(msg: 'Network Error');
        }
        }catch (e) {
Fluttertoast.showToast(msg: e.toString());
        }
    }
}

```

```

    finalList<Widget> _pages = [
HomeScreenContent(),
    //SendAlert(),
AboutPage(), // Ensure ProfilePage is implemented
    ];

```

```

    void _onItemTapped(int index) {
setState(() {
    _selectedIndex = index;
    });
    }
}

```

```

@override
Widget build(BuildContext
context) {    return WillPopScope(
onWillPop: ()async {

```

```

Navigator.push(context, MaterialPageRoute(builder: (context) => HomeScreen(),));
return false;
    },    child: Scaffold(
backgroundColor: Colors.white,
appBar: AppBar(
backgroundColor:
Colors.deepPurple,
    elevation: 0,
    title: Row(
mainAxisAlignment: MainAxisAlignment.start,
    children: [
CircleAvatar(
backgroundImage: NetworkImage(image), // User Image
    ),
SizedBox(width: 10),
    Text("Welcome !\n $name", style: TextStyle(fontSize: 16, fontWeight:
FontWeight.bold,color: Colors.white)),
    ],
    ),
    ),
    body: _pages[_selectedIndex], // Display selected page

bottomNavigationBar:
BottomNavigationBar(    type:
BottomNavigationBarType.fixed,
selectedItemColor: Colors.deepPurple,
unselectedItemColor: Colors.grey,
currentIndex: _selectedIndex, onTap:
_onItemTapped,
    items: [
BottomNavigationBarItem(icon: Icon(Icons.home), label: "Home"),
    //BottomNavigationBarItem(icon: Icon(Icons.taxi_alert), label: "Send Alert"),
BottomNavigationBarItem(icon: Icon(Icons.insert_drive_file_outlined), label: "About"),
    ],
    ),
    ),
    );
}
}

//Extracted Home Screen UI as a separate Widget
class HomeScreenContent extends StatefulWidget
{
    @override
    _HomeScreenContentState createState() =>
    _HomeScreenContentState();
}

```

```

class _HomeScreenContentState extends State<HomeScreenContent> {
    List<String> id_ = <String>[];

```

```
List<String> username_ = <String>[];  
List<String>from_place_ = <String>[];  
List<String> date_ = <String>[];  
List<String> time_ = <String>[];  
List<String>to_place_ = <String>[];  
List<String> phone_ = <String>[];
```

```
String name = 'name';  
String gender = 'gender';  
String email = 'email';  
String phone = 'phone';  
String place = 'place';  
String age = 'age';  
String aadhar_no = 'aadhar_no';  
String liceNo = 'liceNo';  
String image = 'image';
```

```
@override  
void  
initState() {
```

```
super.initState();  
e();
```

```

    load();
    senddata();

}

void senddata() async {
  SharedPreferences sh = await SharedPreferences.getInstance();
  String url = sh.getString('url').toString();
  String lid = sh.getString('lid').toString();
  final urls = Uri.parse(url +
    "view_profile"); try { final response
    = await http.post(urls, body: {
      'lid': lid,
    });
    if (response.statusCode == 200) {
      String status =
        jsonDecode(response.body)['status']; if (status
        == 'ok') {
        setState(() {
          name = jsonDecode(response.body)['name'].toString();
          gender = jsonDecode(response.body)['gender'].toString(); email =
            jsonDecode(response.body)['email'].toString(); place =
            jsonDecode(response.body)['place'].toString(); age =
            jsonDecode(response.body)['age'].toString(); phone =
            jsonDecode(response.body)['phone'].toString(); aadhar_no =
            jsonDecode(response.body)['aadhar_no'].toString(); liceNo =
            jsonDecode(response.body)['liceNo'].toString(); image =
            sh.getString('imgurl').toString() + jsonDecode(response.body)['image'];
        });
      } else {
        Fluttertoast.showToast(msg: 'Not Found');
      }
    } else {
      Fluttertoast.showToast(msg: 'Network Error');
    }
  } catch (e) {
    Fluttertoast.showToast(msg: e.toString());
  }
}

Future<void> load() async {
  List<String> id = <String>[];
  List<String> username = <String>[];
  List<String> from_place = <String>[];
  List<String> date = <String>[];
  List<String> time = <String>[];

  List<String> to_place = <String>[];
  List<String> phone = <String>[];

```

```

try
{
    final pref = await SharedPreferences.getInstance();
    String ip = pref.getString("url").toString();
    String url = ip + "view_others_travel_info";
    var data = await http.post(Uri.parse(url), body:
    {
        'lid': pref.getString("lid").toString(),
    });

    var jsonData = json.decode(data.body);
    var arr = jsonData["data"];

    for (int i = 0; i < arr.length; i++) {
        id.add(arr[i]['id'].toString());
        username.add(arr[i]['username'].toString());
        from_place.add(arr[i]['from_place'].toString());
        date.add(arr[i]['date'].toString());
        time.add(arr[i]['time'].toString());
        to_place.add(arr[i]['to_place'].toString());
        phone.add(arr[i]['phone'].toString());
    }

    setState(() {
        id_ = id;
        username_ = username;
        from_place_ =
        from_place;      date_
        = date;      time_ =
        time; to_place_ =
        to_place;
        phone_ = phone;
    });
    } catch (e) {
        print("Error ----- " + e.toString());
    }
}

@override
Widget build(BuildContext context) {
    return SingleChildScrollView( padding:
    EdgeInsets.all(16), child: Column(
    crossAxisAlignment:
    CrossAxisAlignment.start, children: [
    SizedBox(height: 20),

```

```

GridView.count( crossAxisCount: 3,
    shrinkWrap: true, physics:

```

```

NeverScrollableScrollPhysics(),
children: [
  CategoryItem(icon: Icons.delivery_dining, label: "Parcel info", page:
  ViewParcelinfoPage(title: "")),
  CategoryItem(icon: Icons.abc, label: "Status", page: ViewOwnParcelstatusPage(title:
  "")), CategoryItem(icon: Icons.send, label: "My Request", page:
  ViewMyRequestPage(title: "")), CategoryItem(icon: Icons.password, label: "Change
  Password", page:
  MyUserChangePassword()),
  CategoryItem(icon: Icons.travel_explore, label: "Travel info", page: TravelInfo()),
  CategoryItem(icon: Icons.outlet_rounded, label: "View Request", page:
  ViewOwnRequest()), CategoryItem(icon: Icons.comment, label: "View Complaint",
  page:
  ViewComplaintsPage(title: ")),
    // CategoryItem(icon: Icons.add_alert, label: "Send Alert", page: SendAlert()),
  CategoryItem(icon: Icons.emergency, label: "Emergency Alert view", page:
  EmergencyAlertsPage(title: ")),
  CategoryItem(icon: Icons.request_page, label: "My Request", page: ViewsendRequest()),
  CategoryItem(icon: Icons.feedback, label: "Feedback", page: FeedbackApp()),
  CategoryItem(icon: Icons.person, label: "Profile", page: studProfile(title: ")),
  CategoryItem(icon: Icons.logout, label: "Logout", page: LoginPage()),
    // CategoryItem(icon: Icons.insert_drive_file_outlined, label: "about", page:
  AboutPage()),
  ],
),

SizedBox(height: 20),
],
),
);
}
}

// Category Icon Widget with Navigation
class CategoryItem extends
StatelessWidget {
  final IconData icon;
  final String label;
  final Widget page;

  CategoryItem({required this.icon, required this.label, required this.page});

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: () {
        Navigator.push(
          context,
          MaterialPageRoute(builder: (context) => page),

```

```
    );  
  },  
  child: Column(  
    children: [  
      CircleAvatar(  
        radius: 25,  
        backgroundColor: Colors.deepPurple.withOpacity(0.1),  
        child: Icon(icon, color: Colors.deepPurple, size: 28),  
      ),  
      SizedBox(height: 5),  
      Text(label, style: TextStyle(fontSize: 12)),  
    ],  
  ),  
); } }
```